

Multimedia Retrieval

Chapter 2: Classical Text Retrieval

Dr. Roger Weber, roger.weber@gmail.com

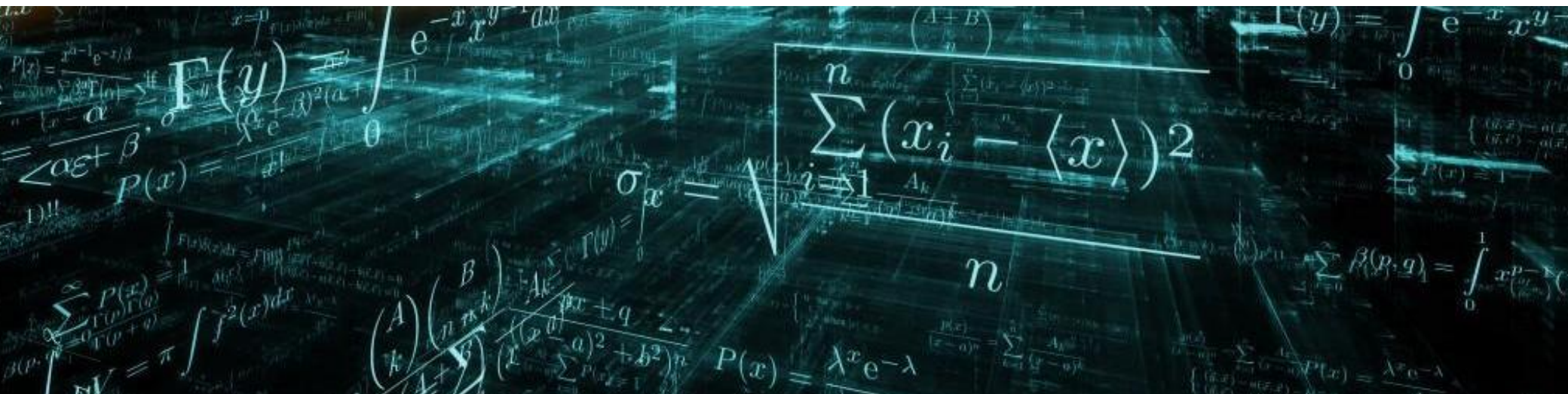
[2.1 Introduction](#)

[2.2 Fundamental Flows](#)

[2.3 Feature Extraction Pipeline](#)

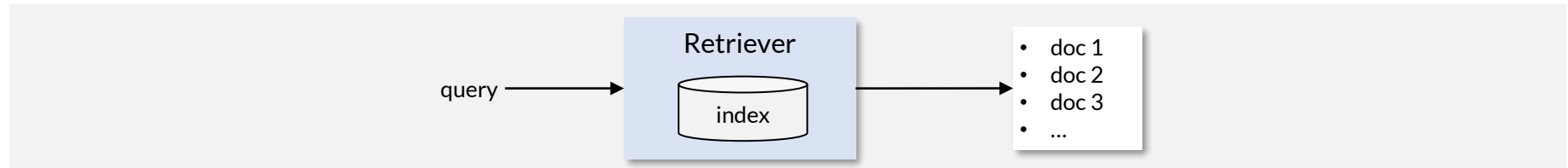
[2.4 Text Retrieval Models](#)

[2.5 Literature and Links](#)



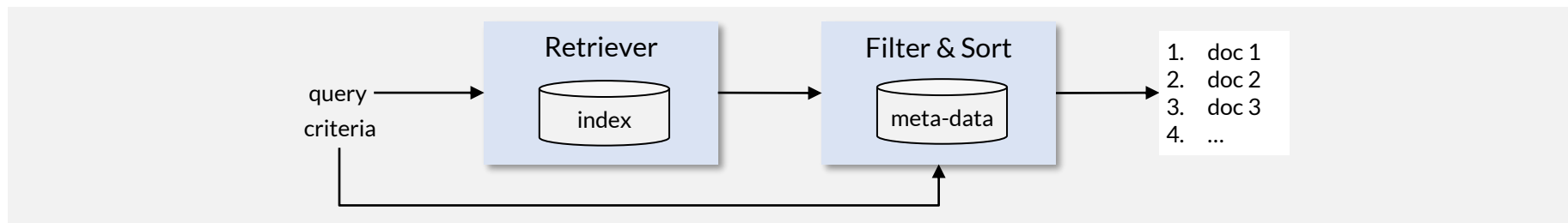
2.1 Introduction

- Text retrieval began in the 1950s and 1960s with pioneering work by Gerard Salton, Karen Sparck Jones, and others. It became popular because it is simple, easy to use, and has many applications. Compared with other media, text retrieval is less affected by the semantic gap. Users enter text queries that systems match to terms in unstructured documents. Other media can benefit when they include textual metadata. For decades this approach was the most efficient way to search multimedia content, but it required obtaining textual metadata.
- Matching terms was simple and adequate for early computer systems. Boolean retrieval soon emerged as a promising method to filter collections for relevant information. The first generation mostly used retriever-only systems that did not explicitly rank documents. For small datasets, this often returned a few pages of useful results.



- Boolean retrieval systems have an advantage: they can determine each document's relevance independently while scanning the data, so no post-processing is needed to sort or rank results. Filters for attributes such as publication date or author are easy to add to the Boolean model. This simple, robust approach still underlies systems today, for example when searching for items on a e-commerce web site.
- The Boolean model uses set theory and Boolean algebra. Each document is represented as a set of terms; term frequency and term proximity are ignored. A query is a Boolean expression that combines simple term-match conditions with operators such as AND and OR. If a document satisfies the expression and any metadata filters, it is included in the result set, otherwise it is excluded. This enables a straightforward mapping of the user's query to a search execution plan.
- Boolean models do not use scoring or ranking, so they can return results as soon as they find the first matching document while scanning the data. For example, when searching a local hard drive this reduces time to the first match. They can also use a simple index called an inverted file, which makes searches very efficient by examining only a small fraction of the data. This approach is still used in modern implementations of retrieval. We will discuss indexing in a later chapter in more details.

- As collections grew, the Boolean model needed an extension to better organize and explore search results. When a query returns hundreds of hits, users need a faster way to navigate them. A post-processing step lets users filter and sort results independently of the query, for example by author or publication date. Unlike retrieval, users can add or remove filters and change sorting while browsing, without resubmitting the search. This feature is called **faceted search** and is common in e-commerce. It does not change document relevance assessments but makes it much easier to explore result sets in an easy to understand and interactive way.



- The method above works well when searches focus mostly on metadata, as in shop or library queries. A major drawback is that sorting does not consider how well an object matches the query. Early extensions of the Boolean model, known as the **Extended Boolean Model**, addressed this by examining the presence and frequency of query terms in documents and using that to assess relevance. For example, consider the query "cat AND dog" and these three documents:

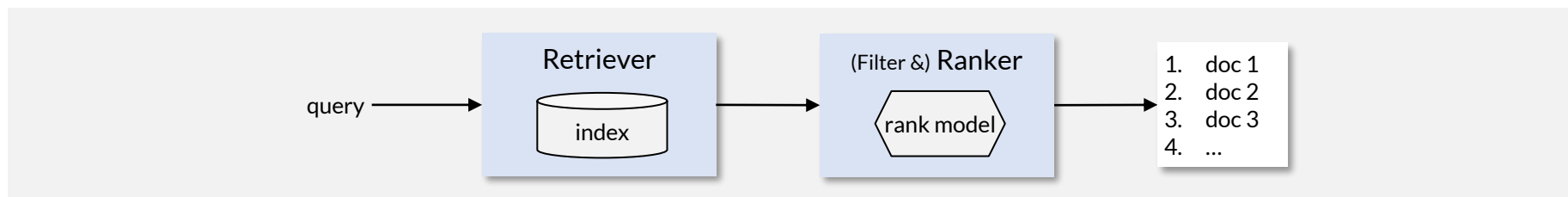
- 1) "A cat walked down the street."
- 2) "The dog chased the cat."
- 3) "The cat played with the dog when another cat and dog approached them."

Documents 2 and 3 satisfy the condition "cat AND dog", but document 1 is rejected by Boolean logic even though it is partially relevant to the query. Moreover, document 3 contains the query terms more often and appears to be a better match, yet the Boolean expression treats documents 2 and 3 as equivalent. The Extended Boolean Model changes the base model in two ways:

- It allows for partial matches to the query (like Document 1) but assigns them lower relevance scores
- It considers how often query terms appear in documents when calculating relevance scores

Using these relevance scores, we can sort the document collection and present results even if not all conditions are met. In other words, instead of using "hard" conditions, we apply penalties for not meeting all conditions.

- In the 1970s, the classical **Vector-Space and Probabilistic retrieval models** appeared. Both set up more elaborated ways to judge how well a document matches a query. In the vector-space model, documents and queries are represented as high-dimensional vectors, and heuristic methods compare those vectors to estimate relevance. Probabilistic models treat documents as produced by a random process and judge relevance by the probability that a document is relevant to the query. Later methods, such as BM25, combine ideas from both approaches.
- Extended Boolean model, Vector Space retrieval, and Probabilistic retrieval follow the same basic process: a retriever selects a set of candidate documents based on the query terms, and a ranking model scores those documents to produce a sorted result list. Additional filters on meta-data, can be applied to narrow the results.



- In this chapter, we begin by examining traditional document representations that support early and modern information retrieval systems. The simplest are the **set-of-words** and **bag-of-words** models. In the set-of-words model, a document is represented as a set of unique terms; both term frequency and word order are discarded. The bag-of-words model records how often each term appears but still ignores term sequence and proximity. In the following, we treat terms as identical to individual words. In later chapters we will refine this notion with additional **linguistic processing** and **phrase detection**, and eventually move to distributed representations and **semantic embeddings** that capture meaning beyond exact word matches.
- We then trace the evolution of retrieval models, from Boolean and Extended Boolean models, through Vector Space and Probabilistic retrieval, to BM25. Each generation improved how we represent documents, estimate relevance, and rank results, and laid the groundwork for the semantic and neural approaches discussed in later chapters.
- This chapter focuses mainly on feature extraction and relevance assessments. Later chapters present fast algorithms that identify candidate sets for sorting, greatly reducing search time. As we will see, the inverted-file method delivers low-latency search results even for collections with billions of documents.

2.2 Fundamental Flows

- For this chapter and the ones that follow, we introduce a simple document structure that lets us explore retrieval models and the implementation of search algorithms. A document consists of three key elements:
 - **Document ID** – ideally a universally unique identifier (UUID) to enable efficient sharding techniques (see the section on distributed indexing).
 - **Metadata attributes** – descriptive fields (e.g., author, publication date, language) used for filtering and faceted search but not included in full-text retrieval.
 - **Content attribute(s)** – the actual text fields indexed for search and ranking.

For simplicity, the models in this chapter assume a single content attribute. Later in this course we will introduce Lucene, which can handle multiple content attributes independently. This allows queries to target one or more attributes, giving additional guidance about which content the user considers relevant.

- To keep the model straightforward, we assume that documents are created after an initial **chunking** step. Chunking splits longer texts into smaller, coherent units. We will study more advanced methods later in the course, but consider a simple example: a PDF file with N pages. Instead of treating the entire PDF as one large document, we split it at the page level and treat each page as an individual (sub-)document. This strategy offers several advantages:
 - **Finer-grained retrieval** – we can not only surface the most relevant PDF files but also highlight the specific page that matches the query, helping users navigate directly to the right passage.
 - **Better precision** – long documents can produce **false positives** because keyword matching ignores term proximity. For example, a query such as “red car” might match a PDF where “red” appears on page 1 and “car” on page 100. Chunking requires query terms to occur near each other (here: within the same page) reducing irrelevant hits.

There is, however, a trade-off. We may miss cases where query terms span adjacent chunks, for instance, one term at the end of page k and another at the beginning of page $k+1$. To reduce these so-called **false negatives**, we can extend each chunk with overlapping text from neighboring chunks. This overlap strategy significantly improves recall. We will revisit overlapping and hierarchical chunking techniques in more detail when discussing **Retrieval-Augmented Generation (RAG)**.

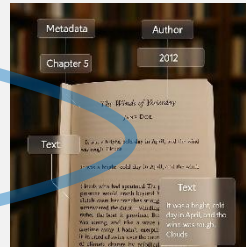
- Many search systems, such as file search on a local drive, scan through the data at query time. This is not efficient for large text collections. Instead, search is split into two parts: an **offline indexing phase** and an **online querying phase** as discussed on the next pages.



b) Split



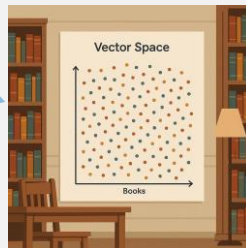
a) Ingest



c) Identify attributes



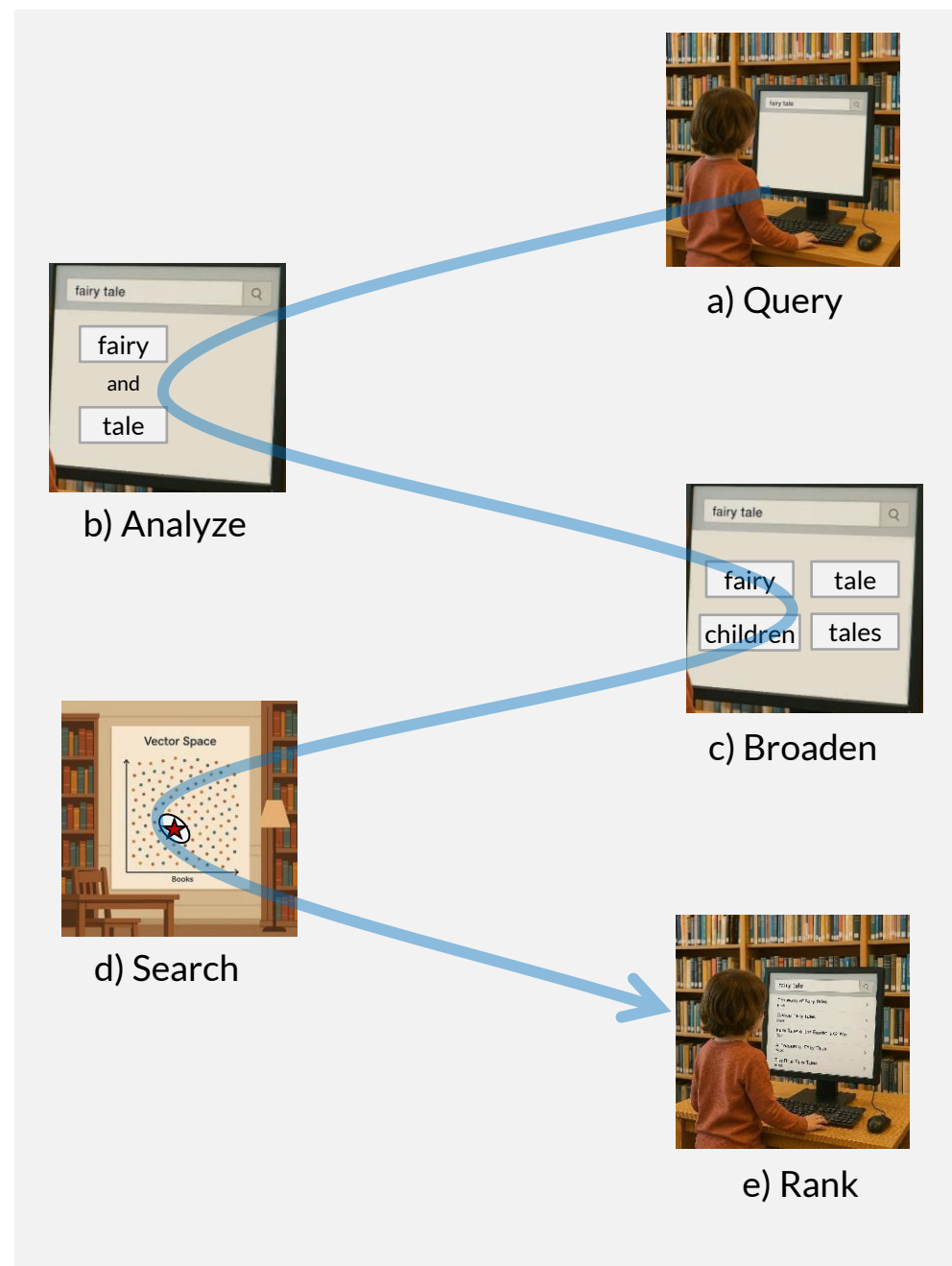
d) Extract + Summarize



e) Build index

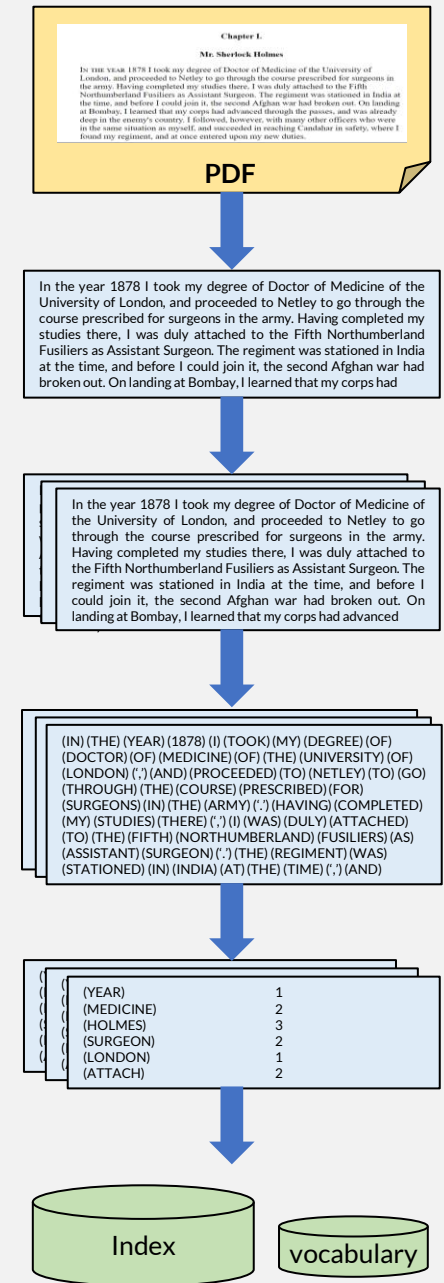
- **Offline Phase:** The diagram on the left illustrates the indexing pipeline: from ingesting a new item to creating the high-dimensional representations used for search. The process unfolds as follows: a new item is added to the collection
 - Ingest the item:** When a new document enters the collection, such as a full book or large report, it is first registered in the system.
 - Split into search units:** The document is divided into smaller, searchable pieces (e.g., pages or sections). Each piece becomes the retrieval granularity, while links back to the parent item allow both page-level and whole-book search.
 - Identify attributes:** For every piece, the system separates **metadata attributes** (title, author, publication year, page number, etc.) from **content attributes** (the actual text to be indexed).
 - Extract and summarize text:** Meaningful tokens are drawn from the content attributes. These tokens are analyzed and transformed into compact feature representations, often high-dimensional vectors capturing the essence of the content.
 - Build the index entry:** The vector representation and the associated metadata are stored together in the index, ready for efficient retrieval.
- The offline phase prepares a searchable index of feature representations and metadata. With this, we can now examine the online phase, where user queries are matched against that index to deliver results.

- **Online Phase:** The figure on the right shows how a user query is processed and matched against the index built during the offline phase. The workflow is as follows:
 - Receive the query:** A user submits a search request, typed, spoken, or captured through handwriting recognition.
 - Analyze and transform the query:** The system tokenizes the query and generates feature representations in the same way it processed documents.
 - Additional steps**, such as spelling correction or synonym expansion, broaden and refine the search.
 - Retrieve matching documents**
The query's feature vector is compared to the vectors stored in the index. Documents with similar representations are treated as potential matches.
 - Rank the results:** Candidate documents are scored often using a retrieval status value (RSV), and sorted so that the most relevant appear first.
- The central challenge in this phase is **relevance ranking**: accurately estimating a document's importance based on the relationship between its feature representation and that of the query. Later chapters will explore advanced techniques, including generative AI, but here we focus on simple, efficient methods suitable for many practical applications.



2.3 Feature Extraction Pipeline

- Before the system can index documents or match user queries, it must turn raw text into a structured, searchable form. The feature-extraction pipeline is the core of the offline phase and moves through four closely linked stages, shown on the right.
- 1. **Extract:** The pipeline begins by ingesting source material in any supported format, such as PDFs, HTML, or scanned text. It isolates relevant content attributes and records metadata (author, title, etc.) for later use.
- 2. **Split:** Long documents are split into smaller, coherent units such as pages, paragraphs, or semantic sections. Each unit, often called a chunk, becomes the basic retrieval unit and is tracked with start and end offsets that map back to the original source.
- 3. **Tokenize:** Each chunk's text is normalized and split into token, words or subwords, using lowercasing, stemming or lemmatization, and stop-word removal. This yields the vocabulary, the set of unique terms across the corpus, which also guides query analysis.
- 4. **Summarize:** Tokens are transformed into feature representations: compact, high-dimensional vectors or weighted term statistics (TF-IDF). Classical forms include the set-of-words, which records only presence of terms, and the bag-of-words, which also includes term frequency.
- The pipeline outputs two key artifacts:
 - **Vocabulary** – the complete set of normalized terms used for both document and query processing
 - **Chunk representations** – high-dimensional feature vectors for every split unit, stored together with source metadata and positional ranges.
- This structured output forms the backbone of the retrieval models in this chapter. We will revisit elements of the pipeline in subsequent chapters to discuss improvements added to the basic versions over time.



2.3.1 Extract Content and Metadata Attributes

- Text documents are available in different formats such as HTML, PDF, EPUB, metadata, or plain text. The first step involves extracting meta information and the sequence of characters that form the text stream without control sequences and formatting information present in the source document. This may include structural analysis of the document, encoding adjustments, and identifying relevant information for feature extraction. In some cases, we may have to apply text extraction from images.
- Consider a simple example in HTML with the following snippet representing a web page's structure. The initial task is to identify the useful bits of information within it. The header typically holds rich meta information, while the body contains the main text parts. Although HTML follows a well-defined standard, extracting information (known as scraping) requires analyzing the data structure used for the pages. In contrast, a web search engine considers everything present on the page.

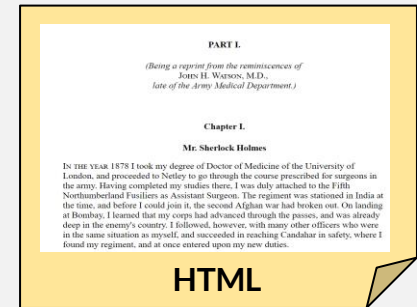
```
<html>
  <head>
    <title> MMIR - 2023 </title>
    <meta name="keywords"
          content="multimedia, retrieval, course"/>
  </head>

  <body>
    ...
    ...
  </body>
</html>
```

Body: Contains the main content enriched with markups. The document's flow is not always obvious and may appear differently on screen than in the file.

Header: Contains meta-information about the document. We can utilize this information to add relevant metadata for the document (and its chunks).

- At this point, we must decide for a character encoding that we will use for the terms and the index, and convert the source text. UTF-8/16/32 are widely used but can limit the ability to support different languages.



In the year 1878 I took my degree of Doctor of Medicine of the University of London, and proceeded to Netley to go through the course prescribed for surgeons in the army. Having completed my studies there, I was duly attached to the Fifth Northumberland Fusiliers as Assistant Surgeon. The regiment was stationed in India at the time, and before I could join it, the second Afghan war had broken out. On landing at Bombay, I learned that my corps had advanced through the passes, and was already deep in the enemy's country. I ...

- Let's use the example of HTML to illustrate some aspects for metadata generation:
 - **URI of page:** both metadata and content (may serve concise key words for retrieval)

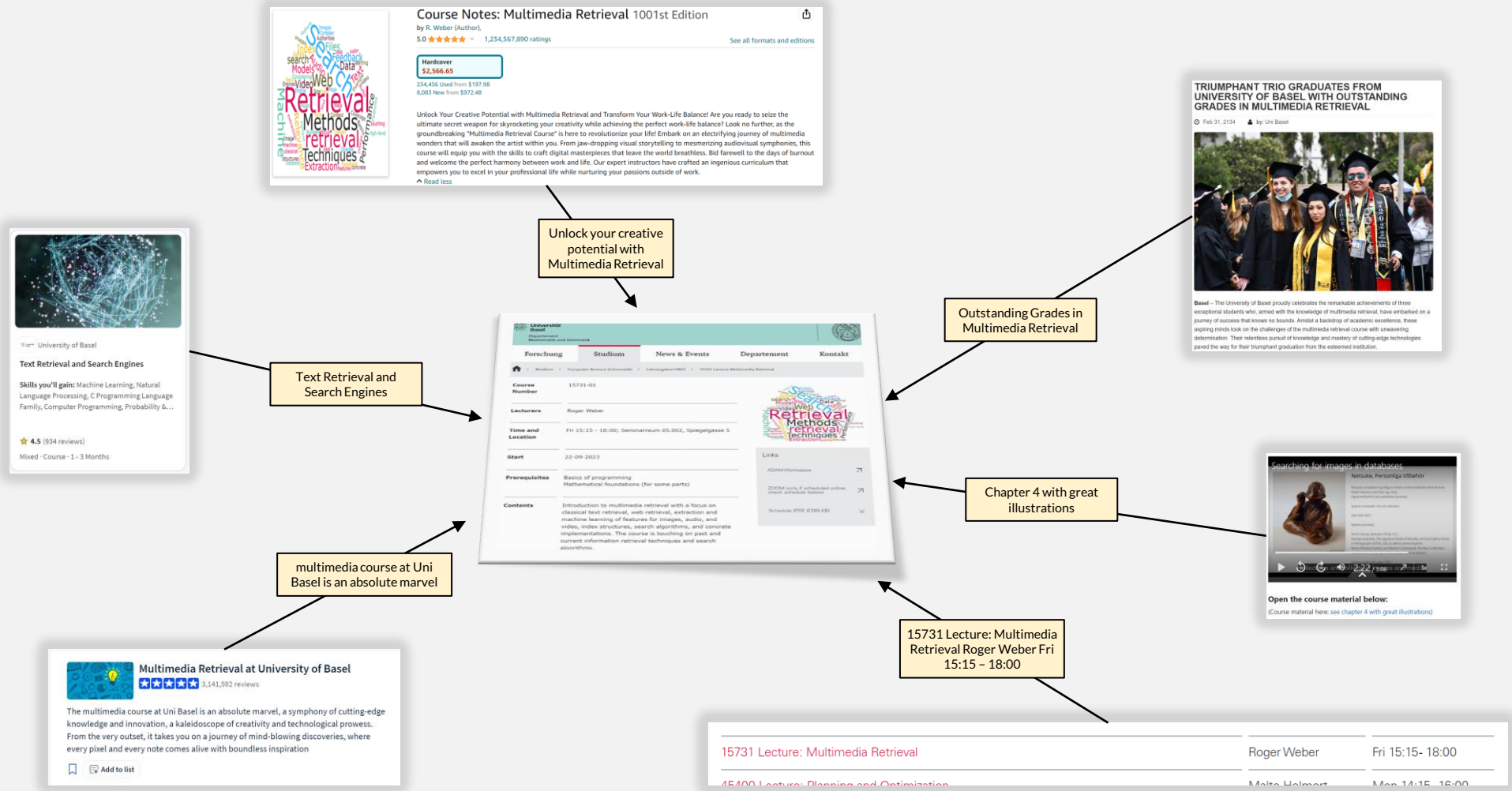

```
https://dmi.unibas.ch/de/studium/computer-science-informatik/lehrangebot-hs23/lecture-multimedia-retrieval/
```
 - **Title of document:** both metadata and content (may serve concise key words for retrieval)


```
<title>Multimedia Retrieval - Homepage</title>
```
 - **Meta information** in header section: (enriched information provided by author)


```
<meta name="keywords" content="MMIR, information, retrieval">
<meta name="description" content="This will change your life...">
```
 - As with all metadata, we must be cautious about its reliability. It might include false information or describe aspects differently from what we observe in other documents. Nevertheless, in many cases, the brief nature of metadata allows us to assign high weights to their text parts.
- Web pages contain links. How do we handle them effectively? Links describe relationships between documents and can enhance the current document's description. More importantly, they also describe the referenced document. Since web page authors often use concise anchor texts, the keywords in anchor texts serve as an excellent source of additional terms for the referenced document. Usually, the link text is associated with both the embedding and linked documents. However, we typically give much higher weight to keywords for the referenced document. It is essential to consider the approach's effectiveness, especially when dealing with click baits (promising more than the referenced documents reveal) or navigational hints like "click here" or "back to the main page". These keywords add no additional content for the referenced document.
- The body includes all text blocks and uses tags to control rendering. The page's flow may not exactly match the order in the HTML file, but it's usually a good enough approximation. Certain tags offer valuable additional information on the following text pieces. For example, we can assign higher weights to term occurrences in headlines, bold text, or text with emphasized rendering on the page.
- HTML includes escape sequences for special characters that need to be translated into the target encoding format.

```
&nbsp; -> space      &uuml; -> ü
```

- The illustration below shows how anchor texts (and their surroundings) provide relevant terms for describing target pages (and images). We emphasized the need for caution with human metadata. However, anchor texts come from diverse sources, simplifying the identification of useful terms across all mentions and filtering out "outliers" with obviously incorrect information. In a subsequent chapter, we will delve into using the link network to assess a page's importance and (objective) relevance through PageRank.



2.3.2 Splitting into Chunks

- Most traditional retrieval methods are designed for short documents. They assign a single term vector to represent an entire document. For small documents, returning the whole file to users is acceptable because they can easily find the relevant spot. For large works like novels, it is necessary to give additional information about where the relevant passage is located. Splitting documents into smaller pieces improves retrieval precision but increases the number of entries in the collection.
- Most traditional retrieval models do not consider term proximity when judging relevance. For example, a query such as "cats AND dogs" could return a novel that uses "cats" on the first page and "dogs" on the last page. Splitting documents into smaller chunks forces terms to appear closer together. In the cats and dogs example, splitting the novel by chapter makes the novel and its chapters less relevant because no chapter contains both "cats" and "dogs".
- There is no one-size-fits-all solution for splitting documents. Generally, it is a trade-off between producing more, smaller but semantically coherent segments and incurring higher storage and retrieval costs:
 - For example, splitting a novel into sentences can create so many entries that performance suffers in a library of thousands of books. Sentences may also be too narrow to produce meaningful matches for more queries.
 - By contrast, a search engine for citations in religious texts can split documents into sentences or verses, creating thousands of small parts that can be retrieved individually.
- The key is to match chunk size to how users search and how they use results. Smaller chunks give more precise matches when users need exact passages, while larger chunks improve performance and provide broader context. In practice, many systems combine both approaches, using overlapping windows, hierarchical chunking, or adaptive methods.

In the year 1878 I took my degree of Doctor of Medicine of the University of London, and proceeded to Netley to go through the course prescribed for surgeons in the army. Having completed my studies there, I was duly attached to the Fifth Northumberland Fusiliers as Assistant Surgeon. The regiment was stationed in India at the time, and before I could join it, the second Afghan war had broken out. On landing at Bombay, I learned that my corps had advanced through the passes, and was already deep in the enemy's country. I ...



In the year 1878 I took my degree of Doctor of Medicine of the University of London, and proceeded to Netley to go through the course prescribed for surgeons in the army. Having completed my studies there, I was duly attached to the Fifth Northumberland Fusiliers as Assistant Surgeon. The regiment was stationed in India at the time, and before I could join it, the second Afghan war had broken out. On landing at Bombay, I learned that my corps had advanced through the passes, and was already deep in the enemy's country. I ...

- In this chapter we briefly outline practical ways to split documents into retrieval units. We will revisit chunking in greater detail, especially for **Retrieval-Augmented Generation (RAG)**, but the following overview highlights common strategies and their trade-offs:

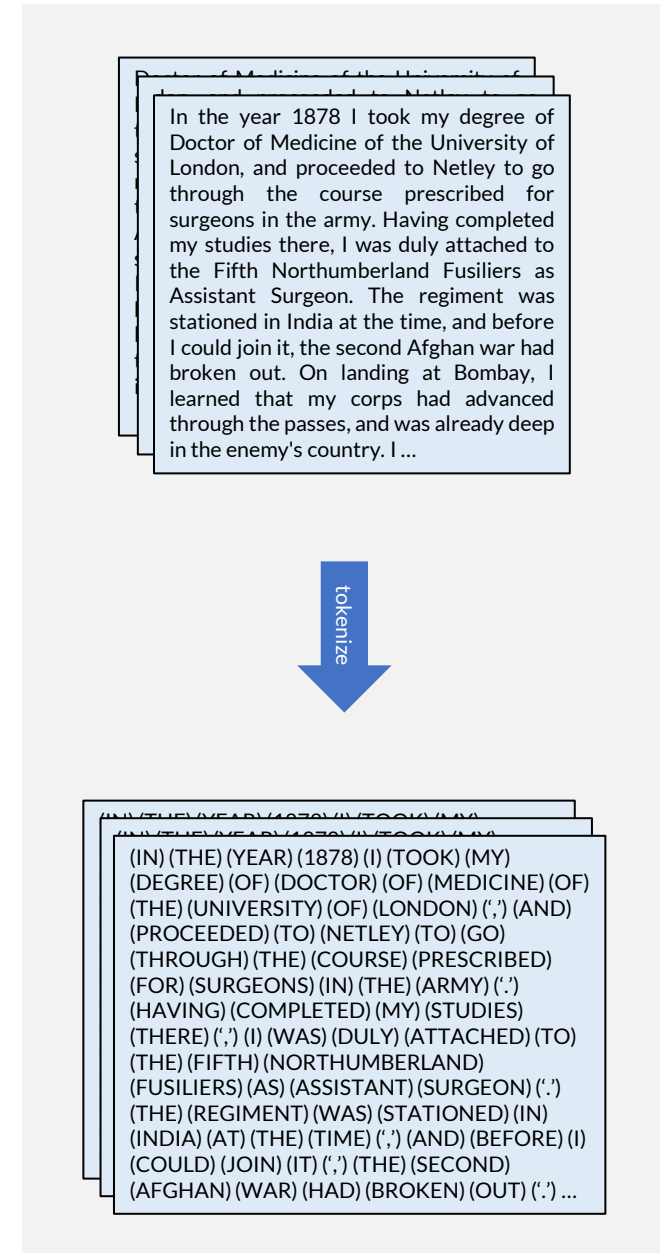
- **Fixed-size chunks:** The text is split into equal segments, each with a fixed number of tokens (words or characters). This method is easy to implement and produces uniformly sized chunks, which simplifies normalization and scoring. However, segment boundaries can split sentences or paragraphs, reducing semantic coherence.
- **Sentence-aware chunks:** Natural-language processing (NLP) techniques first split the text into sentences and then join consecutive sentences until a minimum token threshold is reached. This approach avoids breaking sentences and generally preserves readability. However, it can still split coherent paragraphs or narrative units.
- **Structural or metadata-based chunks:** Use existing structure, such as chapters, sections, pages, or paragraph markers, as natural boundaries. This creates contextually meaningful units and often matches how readers consume the text. However, chunks can vary widely in length, which can complicate ranking.
- These approaches are not mutually exclusive. Many real-world systems combine them: they use structural cues when present, then apply sentence-aligned or fixed-size adjustments.

- In the year 1878 I took my degree of Doctor of Medicine of the University of London, and proceeded to Netley to go through the course prescribed for surgeons in the army. Having completed my studies there, I was duly attached to the Fifth Northumberland Fusiliers as
- Assistant Surgeon. The regiment was stationed in India at the time, and before I could join it, the second Afghan war had broken out. On landing at Bombay, I learned that my corps had advanced through the passes, and was already deep in the
- enemy's country. I followed, however, with many other officers who were in the same situation as myself, and succeeded in reaching Candahar in safety, where I found my regiment, and at once entered upon my new duties. The campaign brought honours and promotion
- to many, but for me it had nothing but misfortune and disaster. I was removed from my brigade and attached to the Berkshires, with whom I served at the fatal battle of Maiwand. There I was struck on the shoulder by a Jezail bullet, which
- shattered the bone and grazed the subclavian artery. I should have fallen into the hands of the murderous Ghazis had it not been for the devotion and courage shown by Murray, my orderly, who threw me across a pack-horse, and succeeded in bringing me safely to
- the British lines. Worn with pain, and weak from the prolonged hardships which I had undergone, I was removed, with a great train of wounded sufferers, to the base hospital at Peshawar. Here I rallied, and had already improved so far as to
- ...

- In the year 1878 I took my degree of Doctor of Medicine of the University of London, and proceeded to Netley to go through the course prescribed for surgeons in the army. Having completed my studies there, I was duly attached to the Fifth Northumberland Fusiliers as Assistant Surgeon.
- The regiment was stationed in India at the time, and before I could join it, the second Afghan war had broken out. On landing at Bombay, I learned that my corps had advanced through the passes, and was already deep in the enemy's country.
- I followed, however, with many other officers who were in the same situation as myself, and succeeded in reaching Candahar in safety, where I found my regiment, and at once entered upon my new duties. The campaign brought honours and promotion to many, but for me it had nothing but misfortune and disaster.
- I was removed from my brigade and attached to the Berkshires, with whom I served at the fatal battle of Maiwand. There I was struck on the shoulder by a Jezail bullet, which shattered the bone and grazed the subclavian artery. I should have fallen into the hands of the murderous Ghazis had it not been for the devotion and courage shown by Murray, my orderly, who threw me across a pack-horse, and succeeded in bringing me safely to the British lines.
- Worn with pain, and weak from the prolonged hardships which I had undergone, I was removed, with a great train of wounded sufferers, to the base hospital at Peshawar. Here I rallied, and had already improved so far as to be able to walk about the wards, and even to bask a little upon the verandah, when I was struck down by enteric fever, that curse of our Indian possessions.
- For months my life was despaired of, and when at last I came to myself and became convalescent, I was so weak and emaciated that a medical board determined that not a day should be lost in sending me back to England. I was dispatched, accordingly, in the troopship "Orontes," and landed a month later on Portsmouth jetty, with my health irretrievably ruined, but with permission from a paternal government to spend the next nine months in attempting to improve it.
- ...

2.3.3 Tokenization

- A token is formed by a sequence of characters. Typically, we use complete words to create tokens, but there are other options which we will explore later in this course. Here's a brief overview:
 - **Characters and fragments of words** can be used to form tokens. For example, breaking the character stream into tokens of 3 characters would turn "street" into "str" and "eet". This method is frequently employed by large language models to maintain a small and constant-sized vocabulary while still being able to encode previously unseen words
 - **Words** are the primary approach used in classical text retrieval. However, we require additional definitions for special characters, numbers, and abbreviations. In certain languages, word boundaries may not always be evident (e.g., Japanese and Chinese). The most significant challenge arises from variations in word forms. For instance, "cat" and "cats" are semantically related, but they are different tokens. **Stemming** is a linguistic method to merge such tokens, enabling better control over vocabulary size and term matching
 - **N-grams and phrases** are composite tokens where multiple words that consistently appear together form a single token. Examples include "San Francisco," "Salt Lake City," "Prime Minister," or "Thai food." While you can manually add such phrases to the vocabulary, we will explore automated methods to detect meaningful phrases or n-grams in the collection later in this course
- In this chapter, we use words as the foundation for studying classical text retrieval methods. In subsequent chapters, we will delve deeper into tokenization and explore various linguistic transformations, along with newer approaches such as embeddings commonly used in generative AI applications.



- Lemmatization and linguistic transformation are essential for matching query terms with document terms, even if they have different inflections or spellings (e.g., "colour" vs. "color"). Depending on the scenario, one or several of the following methods can be used:
 - A common step is stemming. In most languages, words appear in various inflected forms based on time, case, or gender. Examples:

English:	go, goes, went, going, house, houses, master, master's
German:	gehen, gehst, ging, gegangen, Haus, Häuser, Meister, Meisters

As evident from the examples, the inflected forms differ significantly but essentially convey the same meaning. The concept of stemming is to reduce tokens to a common stem and utilize this stem instead. In some languages, like German, stemming is difficult due to its numerous irregular forms and the use of strong inflections ("gehen" → "ging"). In English, Porter defined a very simple algorithm to compute near-stems as explained on the next pages

- Additionally, some languages permit compound words which can result in words of arbitrary length:

German (law in Mecklenburg-Vorpommern, 1999-2013): <i>Rinderkennzeichnungs- und Rindfleischetikettierungsüberwachungsaufgabenübertragungsgesetz</i> (cattle marking and beef labeling supervision duties delegation law)

Finnish: <i>atomiydinenergiareaktorigeneraattorilauhduttajaturbiiniratasvaihde</i> (atomic nuclear energy reactor generator condenser turbine cogwheel stage)
--

In many cases, we aim to break down such compounds to improve the likelihood of matching against query terms. Otherwise, we might never find that German cattle law with a query like "Rind Kennzeichnung." However, breaking a compound may also alter the true meaning of tokens:

German: Gartenhaus → Garten, Haus (ok, not too far away from the true meaning)
German: Wolkenkratzer → Wolke, Kratzer (no, this is completely wrong)

- For English, the Porter Algorithm finds a near-stem of words. This stem is not linguistically correct but it often reduces words with the same linguistic stem to the same near-stem. The algorithm is highly efficient, and various extensions have been proposed over the years. In this context, we focus on Porter's original version from 1980:
 - Porter defines **v** as a „vocal“ if

- it is an **A, E, I, O, U**
- it is a **Y** and the preceding character is not a „vocal“ (e.g. RY, BY)

- All other characters are consonants (**c**)
- Let **C** be a sequence of consonants, and let **V** be a sequence of vocals
- Each word follows the following pattern:

[C](VC)^m[V] **m** is the measure of the word

- further:

- ***o**: stem ends with **cvc**; second consonant must not be W, X or Y (-WIL, -HOP)
- ***d**: stem with double consonant (-TT, -SS)
- ***v***: stem contains a vocal

- The rules on the next pages establish mappings for words using the forms mentioned above. The variable **m** is utilized to prevent over-stemming of short words. Due to limited space, only a few rules are presented here. For a complete set of rules, please refer to one of the many implementations of the Porter algorithm or consult the original paper: **Porter, M.F.: *An Algorithm for Suffix Stripping*. Program, Vol. 14, No. 3, 1980**
- There are 5 main steps with several sub-steps within each. Each (sub-)step includes a list of ordered rules to match the endings of terms. Only the first rule that matches is applied, and the algorithm proceeds to the next (sub-)step. Most sub-steps have only a few rules (less than 10) and not more than 20 rules. The JavaScript implementation comprises around 200 lines of code.
- In subsequent chapters of this course, we will explore more advanced methods

Rule

Examples

Step 1

a)	SS	-> SS
	IES	-> I
	SS	-> SS
	S	->

caresses	-> caress
ponies	-> poni
caress	-> caress
cats	-> cat

b)	(m>0) EED	->EE
	(*v*) ED	->
	(*v*) ING	->

feed	-> feed
plastered	-> plaster
motoring	-> motor

... +5 more rules if 2nd/3rd rule match

c)	(*v*) Y	->I
----	---------	-----

pony	-> poni
------	---------

Step 2

(m>0) ATIONAL	-> ATE
(m>0) TIONAL	-> TION
(m>0) ENCI	-> ENCE
(m>0) IZER	-> IZE

relational	-> relate
conditional	-> condition
valenci	-> valence
digitizer	-> digitize

... +16 more rules

Rule		Examples	
Step 3			
(m>0) ICATE	-> IC	triplicate	-> triplic
(m>0) ATIVE	->	formative	-> form
(m>0) ALIZE	-> AL	formalize	-> formal
... +4 more rules			
Step 4			
(m>1) and (*S or *T)ION	->	adoption	-> adopt
(m>1) OU	->	homologou	-> homolog
(m>1) ISM	->	platonism	-> platon
... +16 more rules			
Step 5			
a) (m>1) E	->	rate	-> rate
(m=1) and (not *O)E	->	cease	-> ceas
b) (m>1 and *D and *L)	-> single letter	controll	-> control

2.3.4 Summarization

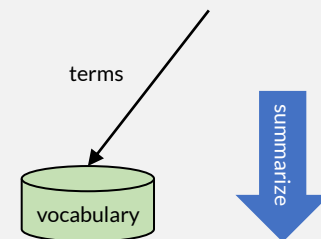
- During summarization, we create concise representations for documents, usually in the form of a high-dimensional feature vector where components represent terms and their occurrences in the documents. To achieve this, we maintain a vocabulary and assign each term a dimension in the feature space.
- To control vocabulary size, we discussed linguistic transformations in the previous action. During summarization, we also evaluate the importance of terms and their ability to describe the content of documents in the collection. The **inverse document frequency (IDF)** is a widely used method to measure the significance of terms. Additionally, we look at stop word elimination as a simpler method of discrimination.
- Once terms are extracted, classical retrieval methods generally use one of two methods to build the feature vector. Let D_i be a document, M be the size of the vocabulary. Then, $d_i \in \mathbb{R}^M$ is its feature representation, and $d_{i,j}$ represents the term t_j in the vocabulary. Additionally, we use $tf(D_i, t_j)$ to denote the number of occurrences of term t_j in document D_i .
 - The **set-of-words** model is a basic representation that only considers whether a term is present or not. It disregards the order of terms, their number of occurrences, and proximity between terms. The feature vector is binary where dimension j indicates the presence of term t_j

$$d_{i,j} = \begin{cases} 1 & tf(D_i, t_j) > 0 \\ 0 & tf(D_i, t_j) = 0 \end{cases} \quad \text{or} \quad d_i = \{t_j \mid tf(D_i, t_j) > 0\}$$

- The **bag-of-words** model is a more common representation and differs from the set-of-words by preserving term frequencies:

$$d_{i,j} = tf(D_i, t_j)$$

(IN) (THE) (YEAR) (1878) (I) (TOOK) (MY)
 (DEGREE) (OF) (DOCTOR) (OF) (MEDICINE) (OF)
 (THE) (UNIVERSITY) (OF) (LONDON) (',' (AND)
 (PROCEEDED) (TO) (NETLEY) (TO) (GO)
 (THROUGH) (THE) (COURSE) (PRESCRIBED)
 (FOR) (SURGEONS) (IN) (THE) (ARMY) (','
 (HAVING) (COMPLETED) (MY) (STUDIES)
 (THERE) (',' (I) (WAS) (DULY) (ATTACHED) (TO)
 (THE) (FIFTH) (NORTHUMBERLAND)
 (FUSILIERS) (AS) (ASSISTANT) (SURGEON) (','
 (THE) (REGIMENT) (WAS) (STATIONED) (IN)
 (INDIA) (AT) (THE) (TIME) (',' (AND) (BEFORE) (I)
 (COULD) (JOIN) (IT) (',' (THE) (SECOND)
 (AFGHAN) (WAR) (HAD) (BROKEN) (OUT) (',' ...



(YEAR)	1
(MEDICINE)	2
(HOLMES)	3
(SURGEON)	2
(LONDON)	1
(ATTACH)	2
(UNIVERSITY)	3
(DULY)	1
...	

- Classical retrieval models treat terms as independent regardless of how close they are syntactically or semantically. For example, "cat" and "cats" are considered different terms. This implies that a query for "cats" will not match documents containing only "cat". To make them match, we need to reduce these forms to the same term as shown with the stemming algorithm during tokenization. The same principle applies to spelling mistakes or variations, either in the document or in the query, e.g., "colour" does not match with "color".
- Controlling the vocabulary size is not primarily a storage or performance concern as we will see with the indexing methods for classical retrieval. Usually, a vocabulary can include millions of terms. However, most documents consist of only a few hundred or thousand terms, depending on how we split them. Consequently, the feature vectors are sparsely populated with non-zero values. Using the **inverted file** method, we store only the non-zero values and during retrieval, we only consider documents that contain a query term.
- However, we notice many terms that are grammatically necessary but do not contribute significantly to the content description. For example, the article "the" in English is one of the most frequent terms in English texts but does not provide relevant information to describe the content. Since almost all English texts contain this article, a search with "the" would retrieve all documents making it unable to differentiate between relevant and non-relevant ones.
- Apart from "the," there are other common **stop words**, as shown in the table on the right with the 50 most frequent terms in a collection D of around 20,000 documents. The second column shows the document frequency $df(t_j)$ which is the number of documents containing the term t_j shown as a percentage of all documents. The last column shows the term frequency of t_j across all documents in the collection D , presented as a percentage of the total number of terms in D (source: <https://faculty.georgetown.edu/wilsong/IR/WD3.html>)
 - The top-50 terms already account for one-third of all terms in the collection, yet they do not significantly contribute to the document description (wasting storage space)
 - All terms appear in more than 60% of the documents, making them unable to distinguish between relevant and non-relevant documents, as they match with most documents
- Stop word lists for most languages are readily available, for example: <https://www.kaggle.com/datasets/heeraldedhia/stop-words-in-28-languages>

t_j	$df(t_j)$	$tf(D, t_j)$
the	100%	6.03%
a	99%	2.57%
of	99%	2.55%
and	99%	2.40%
to	99%	2.58%
in	99%	2.00%
for	98%	0.97%
that	96%	1.15%
on	96%	0.75%
is	95%	0.93%
with	95%	0.70%
at	93%	0.55%
by	92%	0.49%
it	92%	0.69%
as	91%	0.57%
but	91%	0.49%
from	90%	0.44%
be	88%	0.46%
an	88%	0.38%
have	88%	0.45%
was	85%	0.65%
not	84%	0.38%
this	83%	0.35%
are	83%	0.45%
has	83%	0.40%
who	81%	0.37%
they	78%	0.37%
he	78%	0.70%
one	77%	0.26%
said	77%	0.70%
more	75%	0.26%
about	75%	0.27%
or	75%	0.31%
when	74%	0.24%
their	71%	0.27%
his	70%	0.49%
had	70%	0.29%
been	70%	0.21%
all	69%	0.20%
which	69%	0.20%
will	68%	0.27%
out	68%	0.20%
up	68%	0.20%
if	67%	0.21%
than	66%	0.18%
were	66%	0.22%
would	65%	0.23%
can	65%	0.20%
new	64%	0.23%
there	64%	0.18%

- Instead of manually maintaining stop word lists, a more pragmatic approach is based on **Zipf's law**. Let N be the total number of term occurrences (tokens) in the collection and M be the number of distinct terms in the vocabulary. We already used the term frequency $tf(t)$ to denote the number of occurrences of term t . Now, let us order all terms by decreasing term frequencies and assign $rank(t)$ to term t based on that order. The central theorem of Zip's law is that the probability $p(r)$ of randomly selecting the term t with $rank(t) = r$ from the collection is c/r with a constant c that only depends on M as shown on the right side.
- The sum of all $p(r)$ equals 1 and plugging-in the $p(r) = c/r$ for all terms results in a closed formula to estimate c based on the number of terms M . For example, in a collection with $M = 5,000$ different terms, $c = 0.11$, while in a collection with $M = 100,000$, $c = 0.08$.
- The bottom right figure displays the **Zipf distribution** (blue line). As explained earlier, the most frequent words (above the upper cut-off line) hold minimal significance since they appear in nearly every text. The least frequent words (below the lower cut-off) are discriminative but unlikely to appear in queries. The range of meaningful words falls between the lower and upper cut-off points.
- Initially, the idea was to establish cut-off thresholds and exclude words beyond those limits. This would save storage space and enhance search speed. Nowadays, the common practice is to retain all terms, including stop words, but consider the terms' discriminating power (see the red line in the figure) to determine their weight during relevance assessment.
- Consider the search for "it" which is a stop word. If we were to eliminate this term, we would lose the ability to search for IT books or the book "IT" by Stephen King. A query like "the cat" would still search for both terms in documents but would assign significantly higher weight to occurrences of "cat" to determine relevance.

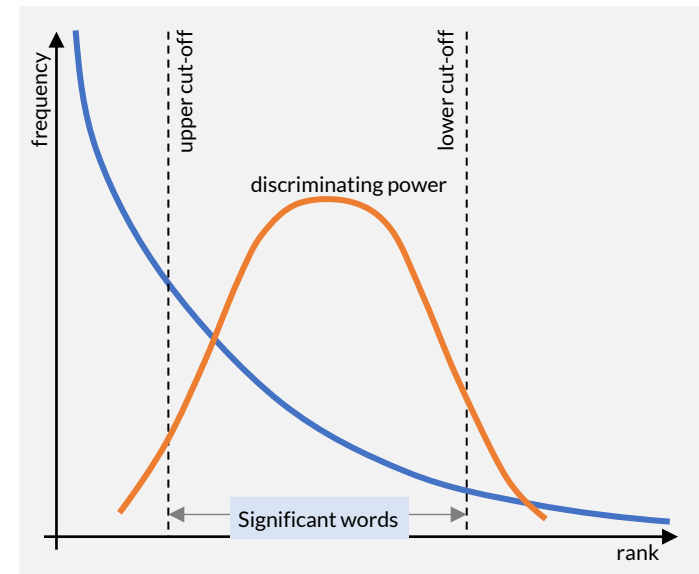
$$p_r = \frac{c}{r} = \frac{tf(t)}{N}, \quad \text{term } t \text{ with } rank(t) = r$$

c is a constant depending only on M

$$1 = \sum_{r=1}^M p_r = \sum_{r=1}^M \frac{c}{r} = c \cdot \sum_{r=1}^M \frac{1}{r}$$

$$c = \frac{1}{\sum_{r=1}^M \frac{1}{r}} \approx \frac{1}{0.5772 + \ln M}$$

M	5'000	10'000	50'000	100'000
c	0.11	0.10	0.09	0.08

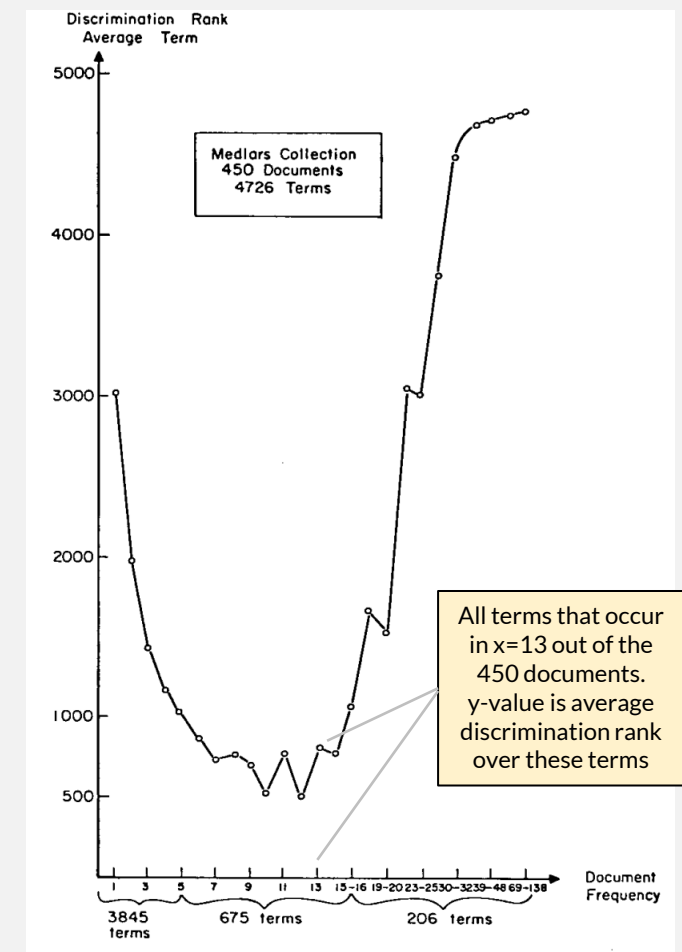


- In their 1975 paper, Salton, Wong, and Yang took a different approach by exploring methods to quantify the discriminatory power of terms. Let's consider a collection with documents D_i and the similarities between them given by $0 \leq \text{sim}(D_i, D_j) \leq 1$. We examine the collection twice, once with the term t in documents and once with it removed, to analyze the impact of the term's presence on similarities. Removing a valuable term from the collection causes documents to become more similar to each other. This is because the valuable term helped to distinguish documents, resulting in lower similarities between them.

- Let $tf(D_i, t_j)$ represent the term frequency of term t_j in document D_i
- We determine the centroid document C by aggregating all M terms with their average frequency $tf(C, t_j)$ across the N documents
- Then, we define the density of the collection as the sum of all similarities between documents and their centroid C :

$$Q = \sum_{i=1}^N \text{sim}(D_i, C) \quad tf(C, t_j) = \frac{1}{N} \cdot \sum_{i=1}^N tf(D_i, t_j) \quad \text{for } \forall j$$

- Finally, we compute the density Q_t for the collection without the term t , and define the discrimination power of term t as: $dp(t) = Q_t - Q$
 - $dp(t)$ is large: if we remove the term t from the collection, similarities to the centroid increase. In other words, the term t differentiates the collection and is hence a significant term
 - $dp(t)$ is negative: if term is present, documents are more similar to the centroid. This can happen, for instance, if a word occurs very frequently in all documents and thus dominates the similarity score
- Sorting terms by their decreasing $dp(t)$ -value assigns a discrimination rank to each term t . The figure on the right illustrates the average ranks (y-axis) for terms occurring in 1, 2, 3, ..., up to 138 documents.



Observation: Terms that appear in very few or numerous documents receive a high discrimination rank. However, terms occurring in 9-12 documents have the smallest discrimination ranks. These terms add significantly to the description of documents in the collection.

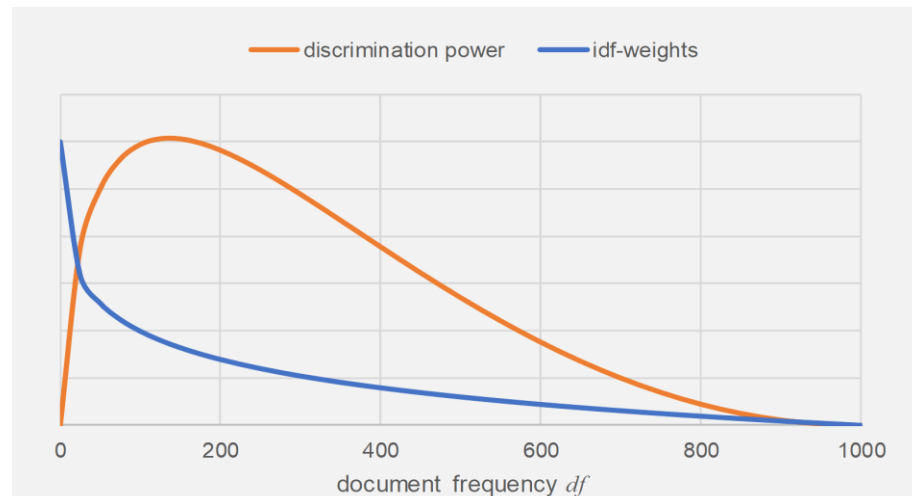
source: Salton, Wong, Yang (1975)

- Karen Spärck Jones (1972) introduced a statistical interpretation for term discrimination called **inverse document frequency (idf)** which has evolved into the standard method for term weighting in relevance assessment. The document frequency $df(t)$ indicates how many documents contain the term t at least once. Let N be the collection's document count. The inverse document frequency $idf(t)$ is expressed as:

$$idf(t) = \log \frac{N + 1}{df(t) + 1} = \log(N + 1) - \log(df(t) + 1)$$

Note that there exist many variants of the idf -formula, but all share the same structure as shown above.

- We can utilize idf to assign weights to components in both query and document feature vectors. As a simplification, let us assume that a term only occurs once in a query. Furthermore, we can estimate the probability that a term t is part of the query to be proportional to $df(t)/N$ (we need to normalize by the sum over all terms to obtain probability values). Finally, the components of the weighted document vector for D_i are given by $idf(t) \cdot tf(D_i, t)$
- Comparing vectors in vector space retrieval relies on the inner vector product. We multiply query and document components and aggregate these values. Consequently, the term's discrimination power approximately equals $idf(t)^2 \cdot tf(D_i, t) \cdot p(t)$ over all queries and documents. This value predicts a term's contribution to the relevance assessment (here for the inner vector product), or in other words, how useful the term is to describe the content and to distinguish between relevant and non-relevant documents.
- The lower right graph depicts idf -weights (blue) and discrimination power (red) based on document frequency df with $N = 1,000$ documents:
 - Terms with low document frequencies (left side) have high idf -weights but are scarcely present in queries, leading to low discrimination power
 - On the right side, terms with high document frequency have both low weights and discrimination power
 - Terms around $df = 100 = 0.1 \cdot N$ exhibit the highest discrimination power



2.4 Text Retrieval Models

- In the upcoming sections, we explore various retrieval models, examining their pros and cons. While we focus on the key methods, it's important to note that there are numerous extensions in literature. Throughout this chapter, we'll employ the following notations:

Notation	Value Range	Description
$\mathbb{D}$	$\{D_1, \dots, D_N\}$	Collection of N documents
D_i		Representation of a document with $1 \leq i \leq N$
$\mathbb{T}$	$\{t_1, \dots, t_M\}$	Collection of M terms
t_j		Representation of a term with $1 \leq j \leq M$
$\mathbf{d}_i$	$\{0,1\}^M, \mathbb{N}^M, \text{ or } \mathbb{R}^M$	Feature description of document D_i with the j -th dimension describing document with regard to term t_j
$\mathbf{A}$	$\{0,1\}^{M \times N}, \mathbb{N}^{M \times N}, \text{ or } \mathbb{R}^{M \times N}$	Term-document matrix with $a_{j,i} = tf(D_i, t_j)$, that is rows denote terms and columns denote documents. For instance, the i -th column is $a_{:,i} = \mathbf{d}_i$.
$tf(D_i, t_j)$	$\mathbb{N}$	Term frequency of term t_j in document D_i , i.e., number of occurrences of term t_j in document D_i
$df(t_j)$	$\mathbb{N}$	Document frequency of term t_j in the collection $\mathbb{D}$, i.e., number of documents in $\mathbb{D}$ that contain term t_j at least once
$idf(t_j)$	$\mathbb{R}$	Inverse document frequency of term t_j given by $idf(t_j) = \log(N + 1) - \log(df(t_j) + 1)$
Q		Representation of a query
$\mathbf{q}$	$\{0,1\}^M, \mathbb{N}^M, \text{ or } \mathbb{R}^M$	Feature description of query Q with the j -th dimension describing query with regard to term t_j
$sim(Q, D_i)$	$[0,1]$	Similarity between query Q and document D_i . 0 means dissimilar, 1 means identical

2.4.1 Standard Boolean Model

- The original Boolean models treated documents and queries as **sets of words**, aiming to find documents containing all query terms. Later, Boolean expression enhanced queries and allowed for more complex search scenarios. A key advantage was the ability to decide for each document whether it is relevant and in the result, independently of the rest of the collection. As such, the **Standard Boolean Model** functions as a filtering predicate selecting relevant items rather than assessing their relevance. Initially, Boolean retrieval focused on data retrieval, lacking the capacity to rank documents by importance. We labeled these as "Retrieval-only" engines.
- Boolean expressions consist of two atomic predicates and two methods for merging them into expressions. The atomic predicates are: 1) presence of a term ('must be present') and 2) absence of a term ('must not be present'). These atomic predicates are then combined using the AND and OR operators to create the query expression.

- $Q = t$ Term t must be present
- $Q = \neg t$ Term t must not be present
- $Q = Q_1 \vee Q_2$ Sub-query q_1 or sub-query q_2 fulfilled
- $Q = Q_1 \wedge Q_2$ Both sub-query q_1 and q_2 fulfilled

- Following the rules for Boolean expressions, we can transform the query expression into a disjunction normal form:

$$Q = (\tau_{1,1} \wedge \cdots \wedge \tau_{1,K_1}) \vee \cdots \vee (\tau_{L,1} \wedge \cdots \wedge \tau_{L,K_L}) = \bigvee_{l=1}^L \left(\bigwedge_{k=1}^{K_l} \tau_{l,k} \right)$$

with $\tau_{l,k} = t_{j(l,k)}$ or $\tau_{l,k} = \neg t_{j(l,k)}$ ($j(l,k)$ is the mapping to the index of the term used in the query)

- Query evaluation can be approached in two ways: 1) individually assess the predicate for every document, and 2) employ set operations to derive the result set from the entire collection:
 - 1) For each document being examined, calculate the values for all $\tau_{l,k}$ based on the presence or absence of query terms in the document, considering whether the term 'must be present' or 'must not be present'. If the evaluation of the disjunctive normal form results in a true value, the document is marked as relevant

- 2) To enhance query evaluation speed, we only need to focus on documents that either contain the query term ('must be present') or don't contain it ('must not be present'). Consequently, for each atomic predicate, we can create sets $\mathbb{S}_{l,k}$ that include precisely the documents that satisfy the atomic predicate:

$$\mathbb{S}_{l,k} = \begin{cases} \{D_i \mid tf(D_i, t_{j(l,k)}) = 1\} & \text{if } \tau_{l,k} = t_{j(l,k)} \\ \{D_i \mid tf(D_i, t_{j(l,k)}) = 0\} & \text{if } \tau_{l,k} = \neg t_{j(l,k)} \end{cases}$$

Following the same structure of the disjunctive normal-form of the query, we use set intersections and unions to compute the final set of relevant documents:

$$\mathbb{Q} = \bigcup_{l=1}^L \bigcap_{k=1}^{K_l} \mathbb{S}_{l,k} = \bigcup_{l=1}^L \bigcap_{k=1}^{K_l} \begin{cases} \{D_i \mid tf(D_i, t_{j(l,k)}) = 1\} & \text{if } \tau_{l,k} = t_{j(l,k)} \\ \{D_i \mid tf(D_i, t_{j(l,k)}) = 0\} & \text{if } \tau_{l,k} = \neg t_{j(l,k)} \end{cases}$$

Later in this chapter, we will introduce the inverted file method, we applies this evaluation scheme to provide fast response times.

- **Advantages:** Simple model with clear query semantics. Easy to implement and user-friendly. Fast evaluation with sets enables quick searches, even for large data sets. Boolean expressions offer precise control for including or excluding documents, influencing result size. This model can explain why a document was considered relevant. Easy to extend with other filtering criteria over metadata of documents (e.g., language = 'English')
- **Disadvantages:** Limited control over result size—users may get too few or too many results. Larger result sets lack ranking, requiring manual browsing. If the set of relevant documents is small, the method does not show 'partial matches', i.e., documents that fulfill some of the atomic predicates but not all. Although the query language is simple, users may find it hard to express a complex information need as a combination of ANDs and ORs. However, to improve the definition of 'what is relevant', users require more complex queries. All terms have the same weight, hence, stop words contribute equally to the result as the more significant terms. The Boolean model resembles data retrieval more than information retrieval. We will consider superior models with ranking that offer similar simplicity and performance.

2.4.2 Extended Boolean Model

- In 1983, Salton et. al. extended the Boolean model to overcome the drawbacks discussed previously:
 - introduce scores for ranking, considering weights for terms and term occurrences for atomic predicates
 - support partial matches, i.e., positive scores for documents that do not fulfill all atomic predicates
- The Extended Boolean Model adopts a bag-of-words approach, assigning normalized vectors ($\mathbf{d}_i$) to documents using term occurrences and inverse document frequency (idf). Normalization ensures values within the vector components range between 0 and 1:

$$d_{i,j} = \min\left(1, \frac{tf(D_i, t_j) \cdot idf(t_j)}{\alpha}\right) \quad \forall j: 1 \leq j \leq M \quad \text{with } \alpha = \max\left(tf(D_i, t_j) \cdot idf(t_j)\right) \quad (\text{or some other value})$$

- However, the query remains a Boolean expression as in the standard model:

$$Q = (\tau_{1,1} \wedge \cdots \wedge \tau_{1,K_1}) \vee \cdots \vee (\tau_{L,1} \wedge \cdots \wedge \tau_{L,K_L}) = \bigvee_{l=1}^L \left(\bigwedge_{k=1}^{K_l} \tau_{l,k} \right)$$

with $\tau_{l,k} = t_{j(l,k)}$ or $\tau_{l,k} = \neg t_{j(l,k)}$ $j(l,k)$ is the mapping to the index of the term used in the query

- Rather than 'true' and 'false', atomic predicates yield a similarity score between 0 and 1, determined by the vector component and the 'must be present' or 'must not be present' predicate:

$$sim(\tau_{l,k}, D_i) = \begin{cases} d_{i,j(l,k)} & \text{if } \tau_{l,k} = t_{j(l,k)} \\ 1 - d_{i,j(l,k)} & \text{if } \tau_{l,k} = \neg t_{j(l,k)} \end{cases}$$

- Using the similarity scores for atomic predicates, we can establish how scores are merged for the AND and OR operators in Boolean expressions. Several common methods exist, a selection of which is provided below:

- **Fuzzy Algebraic:** only works for two operands

$$\begin{aligned} \text{sim}(Q_1 \wedge Q_2, D_i) &= \text{sim}(Q_1, D_i) \cdot \text{sim}(Q_2, D_i) \\ \text{sim}(Q_1 \vee Q_2, D_i) &= \text{sim}(Q_1, D_i) + \text{sim}(Q_2, D_i) - \text{sim}(Q_1, D_i) \cdot \text{sim}(Q_2, D_i) \end{aligned}$$

- **Fuzzy Set:** generalization to K sub-queries is straight forward

$$\begin{aligned} \text{sim}(Q_1 \wedge Q_2, D_i) &= \min\{\text{sim}(Q_1, D_i), \text{sim}(Q_2, D_i)\} \\ \text{sim}(Q_1 \vee Q_2, D_i) &= \max\{\text{sim}(Q_1, D_i), \text{sim}(Q_2, D_i)\} \end{aligned}$$

- **Soft Boolean Operator:** generalization to K sub-queries is straight forward

$$\begin{aligned} \text{sim}(Q_1 \wedge Q_2, D_i) &= (1 - \alpha) \cdot \min\{\text{sim}(Q_1, D_i), \text{sim}(Q_2, D_i)\} + \alpha \cdot \max\{\text{sim}(Q_1, D_i), \text{sim}(Q_2, D_i)\} & 0 \leq \alpha \leq 0.5 \\ \text{sim}(Q_1 \vee Q_2, D_i) &= (1 - \beta) \cdot \min\{\text{sim}(Q_1, D_i), \text{sim}(Q_2, D_i)\} + \beta \cdot \max\{\text{sim}(Q_1, D_i), \text{sim}(Q_2, D_i)\} & 0.5 \leq \beta \leq 1 \end{aligned}$$

- **P-Norm-Model:** distances (p-norm) in the query (sub-)vector space

$$\begin{aligned} \text{sim}\left(\bigwedge_{k=1}^K Q_k, D_i\right) &= 1 - \sqrt[p]{\frac{\sum_k (1 - \text{sim}(Q_k, D_i))^p}{K}} & \text{with } 1 \leq p < \infty \\ \text{sim}\left(\bigvee_{k=1}^K Q_k, D_i\right) &= \sqrt[p]{\frac{\sum_k \text{sim}(Q_k, D_i)^p}{K}} \end{aligned}$$

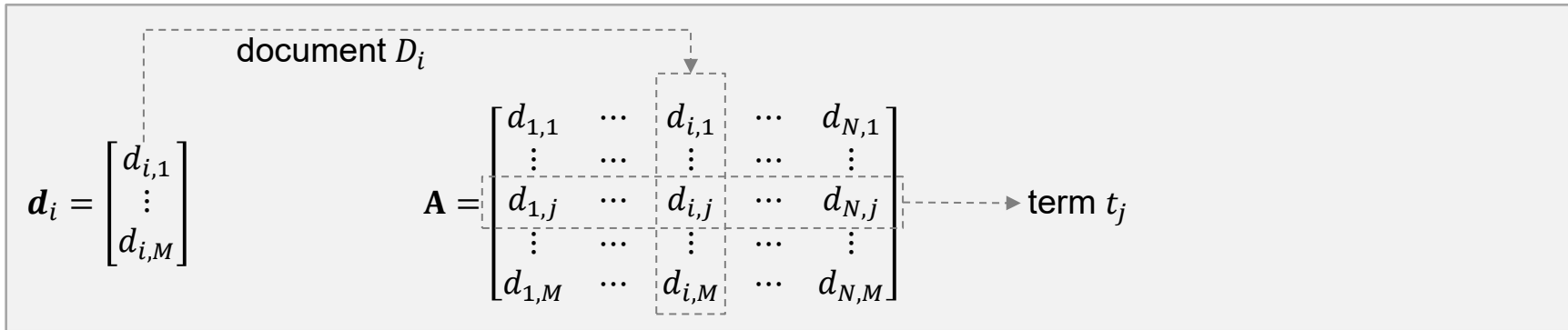
- **Advantages:** simple model with clear query semantics as with standard Boolean model. User-friendly and easy to implement. While query evaluation is heuristic, it offers solid performance. With the inverted file method, similarity values can be efficiently computed. Unlike the standard Boolean model, it provides ranked lists and partial matches, allowing control over result size. Terms are treated differently based on term occurrence and discrimination power.
- **Disadvantages:** Heuristic similarity scores lack clear theoretical explanation. Users might struggle to express complex information needs using the simple query language. Retrieval quality is decent, but other methods with similar computational complexity yield better outcomes.

2.4.3 Vector Space Retrieval

- The initial version of the vector space retrieval model was introduced in the SMART retrieval system by Salton et al. It remains the most widely used classical retrieval model, and we will explore advanced extensions and implementations in this chapter for state-of-the-art retrieval performance.
- Unlike Boolean methods, the vector space retrieval model treats documents and queries as vectors in a high-dimensional feature space. It employs vector-based similarity metrics for ranking. A document D_i is represented as a vector $\mathbf{d}_i$, utilizing idf-weighted term frequencies. Unlike the extended Boolean models, we refrain from normalizing the term frequencies.

$$d_{i,j} = tf(D_i, t_j) \cdot idf(t_j) \quad \forall j: 1 \leq j \leq M$$

- All document representations can be merged into the term-document matrix $\mathbf{A}$. Each column in $\mathbf{A}$ corresponds to a document, and each row represents a term in the vocabulary. Hence, matrix element $a_{j,i} = d_{i,j}$, following the convention of addressing matrix elements by rows and then columns.



- While we illustrate the method in this chapter using the term-document matrix and outline matrix-vector operations for score computation, practical implementations do not store or utilize matrix calculations due to the matrix's sparsity, where many elements are 0 as documents usually have only a few terms. We will explore more efficient evaluation techniques in the subsequent parts of this chapter.

- Queries are depicted as sparse vectors, denoted as $\mathbf{q}$. Unlike Boolean expressions, a query is treated as a mini-document or search prompt, following identical processing steps and vocabulary use as documents. This results in term frequencies in queries, yielding a component q_j through the following method:

$$q_j = tf(Q, t_j) \cdot idf(t_j) \quad \forall j: 1 \leq j \leq M$$

- Various methods exist to compare document vectors with query vectors. In this context, we will discuss the most prominent ones:
 - The **inner vector product** uses the dot-product between the query and document vector. When applied to the entire collection, we multiply the term-document matrix by the query vector and then rank documents based on decreasing similarity values. It is important to note that similarity here is not confined to a range between 0 and 1, and literature often refers to it as retrieval status value (RSV):

$$sim(Q, D_i) = \mathbf{q} \cdot \mathbf{d}_i = \sum_{j=1}^M q_j \cdot d_{i,j}$$

$$\mathbf{sim}(Q, \mathbb{D}) = \begin{bmatrix} sim(Q, D_1) \\ \vdots \\ sim(Q, D_N) \end{bmatrix} = \mathbf{A}^T \mathbf{q}$$

The formula shows that only query terms impact the similarity score, with terms absent in the query yielding a value of 0 for $q_j \cdot d_{i,j}$, irrespective of their frequency in documents. In contrast, documents with larger $d_{i,j}$ values for query terms, that is more term occurrences, receive higher ranks. Notably, significant terms with higher *idf* values have more influence, and this influence is amplified due to *idf* weighting in both queries and documents. Finally, we observe the ‘partial-match’ capability of the model. If a document shares at least one term with the query, then the score is positive.

- The **cosine measure** calculates the angle between document and query vectors. It implies that documents need to contain query terms for high scores. Absence of query terms widens the angle between the vectors, leading to lower scores.

$$sim(Q, D_i) = \frac{\mathbf{q} \cdot \mathbf{d}_i}{\|\mathbf{q}\| \cdot \|\mathbf{d}_i\|} = \frac{\sum_{j=1}^M q_j \cdot d_{i,j}}{\sqrt{\sum_{j=1}^M q_j^2} \cdot \sqrt{\sum_{j=1}^M d_{i,j}^2}}$$

Similar to the inner vector product, scores for all documents can be calculated through matrix-vector multiplications. For this, we normalize the query vector by its size and introduce a diagonal matrix $\mathbf{L}$ with inverse document lengths to dynamically normalize document vectors.

$$\mathbf{sim}(Q, \mathbb{D}) = \begin{bmatrix} \mathbf{sim}(Q, D_1) \\ \vdots \\ \mathbf{sim}(Q, D_N) \end{bmatrix} = \mathbf{L} \mathbf{A}^\top \mathbf{q}' \quad \text{with} \quad \mathbf{L} \in \mathbb{R}^{N \times N} = \begin{bmatrix} \frac{1}{\|\mathbf{d}_1\|} & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & \frac{1}{\|\mathbf{d}_N\|} \end{bmatrix} \quad \text{and} \quad \mathbf{q}' = \frac{\mathbf{q}}{\|\mathbf{q}\|}$$

Alternatively, we could normalize document and query vectors during the extraction step and save normalized versions. This makes the inner vector product and the cosine measure equivalent since vectors have a length of 1. Additionally, similar to the inner vector product, partial matching capability is achieved, and terms absent from the query do not affect the search order. However, the cosine measure is less affected by term occurrences compared to the inner vector product due to normalization.

- For a simplified visualization of vector space retrieval, documents are projected into the smaller query vector space spanned by the query terms, while other dimensions have no effect on search order:
 - Using the inner vector product, a hyperplane through the origin is established with the query vector as its normal. Documents farther from this plane are considered more relevant
 - On the other hand, the cosine measure creates hyper-cones with the query vector as their axis. Higher cosine values correspond to smaller angles of a hyper-cone embedding the document
 - Documents lacking query terms are placed at the origin, yielding a value of 0 with both measures. This allows us to disregard such documents and focus on those containing at least one query term. This leads to efficient retrieval methods explored later using inverted files.
 - An issue arises when query terms are similar (e.g., 'house' and 'villa'), as they might not affect results unless pre-processing merges them. This limitation is common in classical retrieval techniques, often addressed by automatically expanding queries with related terms.

- **Example:** Let's examine a simple collection of three documents to understand the method:

D_1	Shipment of gold damaged in a fire
D_2	Delivery of silver arrived in a silver truck
D_3	Shipment of gold arrived in a truck

Q	gold silver truck
-----	--------------------------

- We extract terms, find document frequencies and inverse document frequencies. The document and query are represented as vectors ($N = 3, M = 11$) as follows:

j	Term t_j	$df(t_j)$	$idf(t_j)$
1	a	3	0
2	arrived	2	.176
3	damaged	1	.477
4	delivery	1	.477
5	fire	1	.477
6	gold	2	.176
7	in	3	0
8	of	3	0
9	silver	1	.477
10	shipment	2	.176
11	truck	2	.176

To simplify, we use: $idf(t_j) = \log(N) - \log(df(t_j))$

d_1	d_2	d_3	q
	.176	.176	
.477			
	.477		
.477			
.176		.176	.176
	.954		.477
.176		.176	
	.176	.176	.176

$\underbrace{\hspace{15em}}_{\mathbf{A}}$

with inner
vector product

$$\mathbf{sim}(\mathbf{Q}, \mathbb{D}) = \begin{bmatrix} .031 \\ .486 \\ .062 \end{bmatrix}$$



$$D_2 > D_3 > D_1$$

- **Advantages:** Extremely simple and intuitive query model. Term weights have a good impact on the scores and differentiate between query terms, e.g., reducing the impact of stop words in the query. Easy to implement and highly efficient in calculation. Outperforms Boolean models and can rival top retrieval methods. Naturally supports partial match queries, and documents do not have to include all query terms for high similarity values.
- **Disadvantages:** heuristic similarity scores with little intuition why they work well (no theoretic background for the model). The similarity measures are not robust and can be biased by authors (spamming of terms). If documents are of different lengths, scores can vary significantly due to the higher term occurrences in larger documents. Main assumption of retrieval model is independence of terms which may not hold true in typical scenarios (see synonyms and homonyms).

2.4.4 Probabilistic Retrieval

- The primary criticism of the existing models lies in their heuristic nature. While they perform well, their correctness lacks a solid foundation. Probabilistic retrieval provides a formal approach based on probabilities. $P(R|D_i)$ is the probability that a document D_i is relevant for a query Q , and $P(NR|D_i) = 1 - P(R|D_i)$ is the probability that it's not relevant. The similarity value between query Q and document D_i is then defined as:

$$\text{sim}(Q, D_i) = \frac{P(R|D_i)}{P(NR|D_i)} = \frac{P(R|D_i)}{1 - P(R|D_i)}$$

- The Binary Independence Model (BIR) is a straightforward approach grounded in several key assumptions for calculating the mentioned conditional probabilities. These assumptions are as follows:

1. Term frequency does not matter (utilizing a set-of-words document model)
2. Term independence (consistent with previous models)
3. Terms absent from the query do not influence ranking (if a term is absent from the query, it's assumed to be equally distributed among relevant and non-relevant documents)

- Given these assumptions, our next step is to derive a closed formula for the similarity scores. To begin, we apply Bayes' theorem to the conditional probabilities above:

$$\text{sim}(Q, D_i) = \frac{P(R|D_i)}{P(NR|D_i)} = \frac{P(D_i|R) \cdot P(R)}{P(D_i|NR) \cdot P(NR)}$$

These new probabilities can be interpreted as follows: $P(R)$ and $P(NR)$ represent the probabilities that a randomly selected document is relevant and not relevant, respectively. $P(D_i|R)$ and $P(D_i|NR)$ are the probabilities that document D_i belongs to the set of relevant and non-relevant documents, respectively.

- Now, leveraging the assumption of binary document vectors and term independence:

Assumption 1:
Documents are
binary vectors

Assumption 2: Terms
are independent

Assumption 1: Documents
are binary vectors

$$P(D_i|R) = P(d_i|R) = \prod_{j=1}^M P(d_{i,j}|R) = \prod_{\forall j: d_{i,j}=1} P(d_{i,j} = 1|R) \cdot \prod_{\forall j: d_{i,j}=0} P(d_{i,j} = 0|R)$$

$$P(D_i|NR) = P(d_i|NR) = \prod_{j=1}^M P(d_{i,j}|NR) = \prod_{\forall j: d_{i,j}=1} P(d_{i,j} = 1|NR) \cdot \prod_{\forall j: d_{i,j}=0} P(d_{i,j} = 0|NR)$$

- Let's use a compact notation for the conditional probabilities in the formula above. Define:

$$r_j = P(d_{i,j} = 1|R) \quad n_j = P(d_{i,j} = 1|NR)$$

r_j is the probability of a relevant document having the term t_j , and n_j is the probability of a non-relevant document having the term t_j . Using this notation, we can express the similarity value in simpler terms:

$$sim(Q, D_i) = \frac{P(R)}{P(NR)} \cdot \prod_{\forall j: d_{i,j}=1} \frac{r_j}{n_j} \cdot \prod_{\forall j: d_{i,j}=0} \frac{1 - r_j}{1 - n_j}$$

$$sim(Q, D_i) \sim \prod_{\forall j: d_{i,j}=1} \frac{r_j}{n_j} \cdot \prod_{\forall j: d_{i,j}=0} \frac{1 - r_j}{1 - n_j}$$

It is important to observe that there is no need to calculate $P(R)$ and $P(NR)$ as they are solely determined by the query and do not affect the document ranking as they linearly scale the similarity values. Therefore, the simplified lower formula produces the same document ranking as the original upper formula.

- We conclude by applying the third assumption: if term t_j is absent in the query, we assume $r_j = n_j$, i.e., non-query terms occur with equal probability in relevant and non-relevant documents. As a result, when $q_j = 0$, the ratios r_j/n_j and $(1 - r_j)/(1 - n_j)$ become 1, and can be omitted from the calculations:

Assumption 3: non-query terms do not impact result

$$\text{sim}(Q, D_i) \sim \prod_{\forall j: d_{i,j}=1} \frac{r_j}{n_j} \cdot \prod_{\forall j: d_{i,j}=0} \frac{1-r_j}{1-n_j} = \prod_{\forall j: d_{i,j}=1, q_j=1} \frac{r_j}{n_j} \cdot \prod_{\forall j: d_{i,j}=0, q_j=1} \frac{1-r_j}{1-n_j}$$

We remove the condition $d_{i,j} = 1$ from the second product and need to adjust in the first product:

$$\text{sim}(Q, D_i) \sim \prod_{\forall j: d_{i,j}=1, q_j=1} \frac{r_j \cdot (1 - n_j)}{n_j \cdot (1 - r_j)} \cdot \prod_{\forall j: q_j=1} \frac{1 - r_j}{1 - n_j}$$

Next, we remove the second product, which solely depends on the query, and linearly scales the similarity values:

$$\text{sim}(Q, D_i) \sim \prod_{\forall j: d_{i,j}=1, q_j=1} \frac{r_j \cdot (1 - n_j)}{n_j \cdot (1 - r_j)}$$

Finally, we arrive at a simple similarity function as a sum of c_j -values. It is important to note that we only need to calculate c_j for query terms, which as with other models so far greatly boost query evaluation with inverted files:

$$\text{sim}(Q, D_i) \sim \sum_{\forall j: d_{i,j}=1, q_j=1} c_j \quad \text{with } c_j = \log \frac{r_j \cdot (1 - n_j)}{n_j \cdot (1 - r_j)}$$

- To calculate the c_j -values, the BIR model starts with initial estimates for a first result list, and then refines these estimates based on user feedback to enhance the results. With ongoing user input on relevant and non-relevant documents in the outcomes, we can iteratively adjust the estimates and offer better outcomes.
 - We introduced r_j and n_j as the probabilities that a relevant and non-relevant document contains the term t_j , respectively. With the user's relevance assessment, we now possess subsets of relevant and non-relevant documents, which allow us to estimate these probabilities by counting the occurrences of term t_j in these subsets
 - **Initial Estimates:** In the absence of feedback, we assume that query terms are more likely to appear in relevant documents, and in non-relevant documents they follow their document frequency. The following estimates are used initially to compute the c_j -values (n_j includes smoothing)

$$r_j = 0.5, \quad n_j = \frac{df(t_j) + 0.5}{N + 1} \quad \forall j: q_j = 1$$

- **Estimates with Feedback:** in each iteration, we ask the user to rate the K retrieved documents and annotate them with relevant (R) and non-relevant (NR). Let L be the number of documents that the user marked as relevant. Further let k_j be the number of retrieved documents that contain the term t_j (that is the document frequency of t_j over the set of retrieved documents), and let l_j be the number of retrieved and relevant documents that contain the term t_j (that is the document frequency of t_j over the set of retrieved and relevant documents). With that, we can estimate new values for r_j and n_j by counting:

$$r_j = \frac{l_j + 0.5}{L + 1}, \quad n_j = \frac{k_j - l_j + 0.5}{K - L + 1} \quad \forall j: q_j = 1$$

We employ the values 0.5 and 1 in the formula above to avoid numerical problems (0-divisions). When no feedback is given, with $L = l_j = 0$, we can set $K = N$ and $k_j = df(t_j)$ to justify the initial estimates.

- The more user feedback we gather, the more accurate the estimates for r_j and n_j become. However, users might be reluctant to provide feedback.

- **Advantages:** The BIR model establishes similarity values on a probabilistic basis through basic assumptions. Document ranking depends on the likelihood of being relevant for the query. Only query terms are necessary for similarity calculations, and the inverted file method offers efficient evaluation. The model performs well, especially after some feedback iterations. It also accommodates partial match queries, where not all query terms need to appear in relevant documents.
- **Disadvantages:** The basic assumptions of the BIR model may not always be valid. As mentioned in the vector space model, term independence is not universally applicable. More complex probabilistic models address term dependence, but they can bring extra computational complexity. Additionally, the document ranking in BIR doesn't consider term frequencies or the discrimination power of terms. Finally, not all users are willing to assist the system with feedback to improve the search results.

2.4.5 Okapi Best Match 25 (BM25)

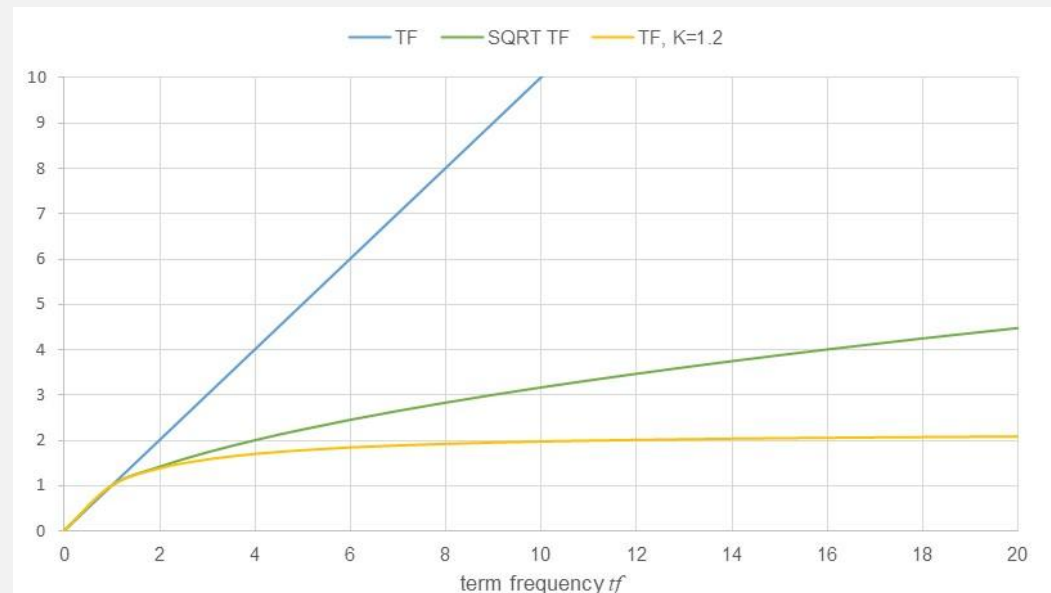
- The Okapi BM25 ranking function was developed at London's City University and is rooted in Karen Spärck Jones' probabilistic framework from the 1970s and 1980s. It is notably applied in Lucene, the engine behind Solr, Elasticsearch, and OpenSearch—three widely used systems for observability, security analytics, and full-text search. BM25 builds on the vector space model as discussed before enhancing it with a probabilistic approach to enhance relevance evaluation.
- Some limitations in the previously discussed models stem from heuristic approaches to identify relevant documents. Researchers developed better frameworks for relevance assessment, driven by key observations:
 1. **Query Term Significance:** the presence or absence of query terms is crucial for relevance assessment
 2. **Partial Matches:** not all relevant documents contain every query term
 3. **Document Length:** longer documents have more terms, but shorter relevant ones should score well too
 4. **Term Specificity:** rare words often carry more meaning than common ones
 5. **Term Saturation:** while term frequency matters, overly frequent terms should not dominate
 6. **Fine Tuning:** flexibility to adjust ranking based on search context
 7. **Efficiency:** efficient retrieval and relevance assessment are essential
 8. **User Feedback:** if available, integration of user feedback for improved search quality
 9. **Term Proximity:** closeness of query terms in a document may indicate higher semantic relevance
 10. **Term Dependence:** recognizing term dependencies, like matching query 'animals' to 'cats' or 'dogs' in documents
- BM25 addresses these observations or provides ways to consider them. We will cover **Efficiency** in the upcoming section on indexing structures and explore **Term Proximity** and **Term Dependence** in the next chapter, where we delve into natural language processing methods.

- Term frequencies play a crucial role in determining document relevance. Typically, we assume that a document's relevance is linked to the frequency of query term occurrences within it. This notion lead to the creation of the $tf \cdot idf$ vector component description. Nonetheless:
 - A document with the search term 'cat' occurring a hundred times is certainly relevant, but it should not be considered twice as relevant as a document with 50 occurrences of 'cat'. In essence, the linear factor tf exaggerates the relevance. It also makes the method vulnerable to spamming attacks
 - Shorter documents have fewer occurrences of terms compared to much longer documents. However, they can be equally or even more relevant. Yet, the $tf \cdot idf$ scheme tends to favor longer documents with higher term frequencies. Very long documents covering a broad range of topics may appear relevant due to their numerous occurrences but users may find it difficult to easily extract the relevant pieces
- A simple adjustment like using $\sqrt{tf}$ instead of tf does not provide significant improvement. We require a function that levels off after a certain occurrence threshold. With $\sqrt{tf}$ we could still influence scoring with excessive spamming of potential query terms.
 - Initially disregarding document length, we can saturate term frequencies as follows:

$$\widehat{tf}_k = \frac{tf \cdot (k + 1)}{tf + k}$$

(k+1) scales values but does not impact ranking

- Typically, $k \in [1, 2]$ with Lucene using $k = 1.2$
- As depicted in the figure on the right, the updated values $\widehat{tf}_k$ saturate relatively swiftly to the value 2.2 with $k=1.2$, whereas unsaturated tf and $\sqrt{tf}$ values increase without limit
- In essence, k serves as a hyperparameter that enables adjustment of the impact of term occurrences on the scoring
- Note: Lucene uses $tf/(tf + k)$ omitting the scaling factor $(k + 1)$ in the numerator



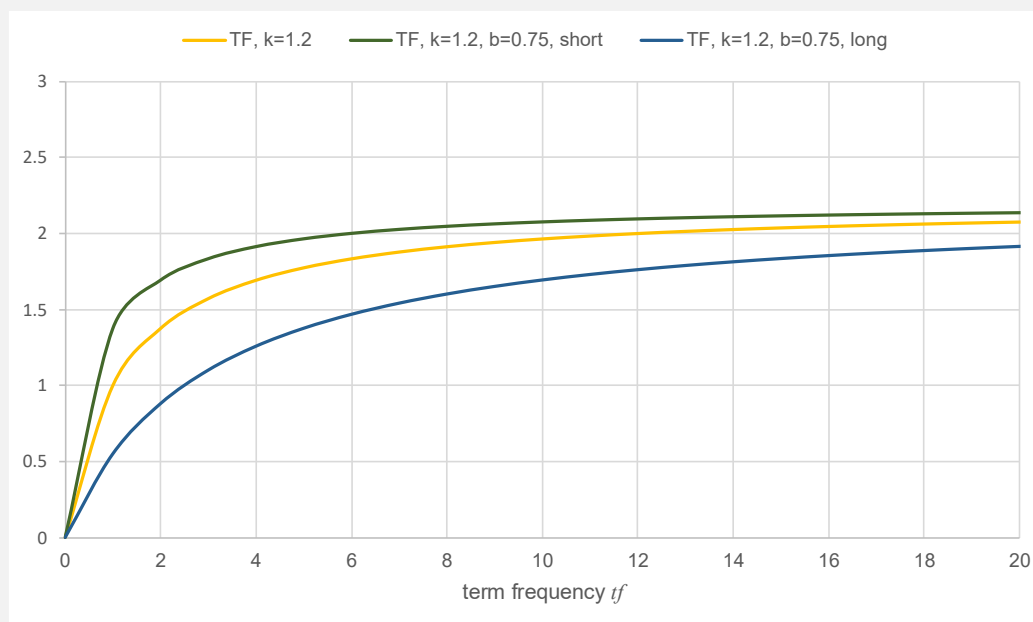
- Now, let's examine document length. Lengthier documents include more terms and should saturate at a slower rate than shorter ones that might not have as many terms. The cosine measure tackled this by normalizing vectors by their length, and then utilizing the inner vector product to determine the angle between the vectors, which remains unaffected by document length. However, BM25 takes a different approach. It employs a summation across all query terms, similar to the inner vector product, while modifying the core formula to account for document length:

$$\widehat{tf}_k(D) = \frac{tf \cdot (k + 1)}{tf + k \cdot \left(1 - b + b \frac{|D|}{adl}\right)}$$

adl is the average document length

with $b = 0.75$ (adjustable), $|D|$ the length of document D , and adl the average length of documents in the collection

- If $|D|$ is smaller than adl (short document), then $\left(1 - b + b \frac{|D|}{adl}\right) < 1$ and values $\widehat{tf}_k(D)$ saturate faster
- If $|D|$ is large (long document), then $\left(1 - b + b \frac{|D|}{adl}\right) > 1$ and values $\widehat{tf}_k(D)$ saturate slower
- $b \in [0,1]$ is a new hyperparameter that steers the impact of document length. Higher values prefer shorter documents
- In the plot to the right, we compare $\widehat{tf}_k$ (graph in the middle) with $\widehat{tf}_k(D)$ of a short document (graph at the top) and with $\widehat{tf}_k(D)$ of a long document (graph at the bottom)
- The difference between shorter and longer documents is significant at lower frequencies but soon diminishes as values saturate to 2.2 for $k = 1.2$
- adl does not have to be the accurate average length of documents. Rather, we can consider it as another hyperparameter to define what 'long' / 'short' means



- We previously discussed *idf*-weights without providing a rationale for using that specific formula. BM25 approaches term weighting probabilistically. Previously, we derived a term weighting function using the BIR-model:

$$c_j = \log \frac{r_j \cdot (1 - n_j)}{n_j \cdot (1 - r_j)} \quad r_j = \frac{l_j + 0.5}{L + 1} \quad n_j = \frac{k_j - l_j + 0.5}{K - L + 1} \quad \forall j: q_j = 1$$

- We introduced r_j and n_j as the probabilities of the term t_j occurring in relevant and non-relevant documents based on user-provided relevance feedback. The calculation of r_j takes into account the number of relevant documents (l_j) out of the L retrieved ones that contain the query term, while n_j considers the number of non-relevant documents ($k_j - l_j$) out of the $K - L$ retrieved ones that contain the query term.
- The BIR model summed up c_j values for the binary document representation. However, c_j values can also serve as weights for terms in the vector space model. We achieve this by incorporating r_j and n_j into the c_j formula:

$$c_j = \log \frac{r_j \cdot (1 - n_j)}{n_j \cdot (1 - r_j)} = \log \frac{\frac{l_j + 0.5}{L + 1} \cdot \left(1 - \frac{k_j - l_j + 0.5}{K - L + 1}\right)}{\left(1 - \frac{l_j + 0.5}{L + 1}\right) \cdot \frac{k_j - l_j + 0.5}{K - L + 1}} = \log \frac{\frac{l_j + 0.5}{L + 1} \cdot \frac{K - L + 1 - k_j + l_j + 0.5}{K - L + 1}}{\frac{L + 1 - l_j - 0.5}{L + 1} \cdot \frac{k_j - l_j + 0.5}{K - L + 1}} = \log \left(\frac{l_j + 0.5}{L - l_j + 0.5} \cdot \frac{K - L - k_j + l_j + 0.5}{k_j - l_j + 0.5} \right)$$

- When user feedback is absent, L and l_j are 0, and we assume that all documents are non-relevant (until proven otherwise) and assign $K = N$ (number of documents) and $k_j = df(t_j)$ (documents containing the term). Substituting these values into the c_j formula results in:

$$c_j = \log \left(\frac{0 + 0.5}{0 - 0 + 0.5} \cdot \frac{N - 0 - df(t_j) + 0 + 0.5}{df(t_j) - 0 + 0.5} \right) = \log \frac{N - df(t_j) + 0.5}{df(t_j) + 0.5}$$

- These c_j values are then used by the BM25 model to refine the initial *idf*-values we discussed earlier. Note that for terms t_j that appear in over 50% of the documents, the logarithm yields a negative value.

$$idf_{BM25}(t_j) = \log \frac{N - df(t_j) + 0.5}{df(t_j) + 0.5}$$

- Let us compare the original *idf*-function (IDF+1) with this new one. The graph on the right displays them against the document frequency in a collection of 1000 documents.
- Additionally, we included the *idf*-function used by Lucene, which incorporates a '+1' term in the logarithm to avoid negative *idf*-values:

$$idf_{lucene}(t_j) = \log \left(1 + \frac{N - df(t_j) + 0.5}{df(t_j) + 0.5} \right)$$

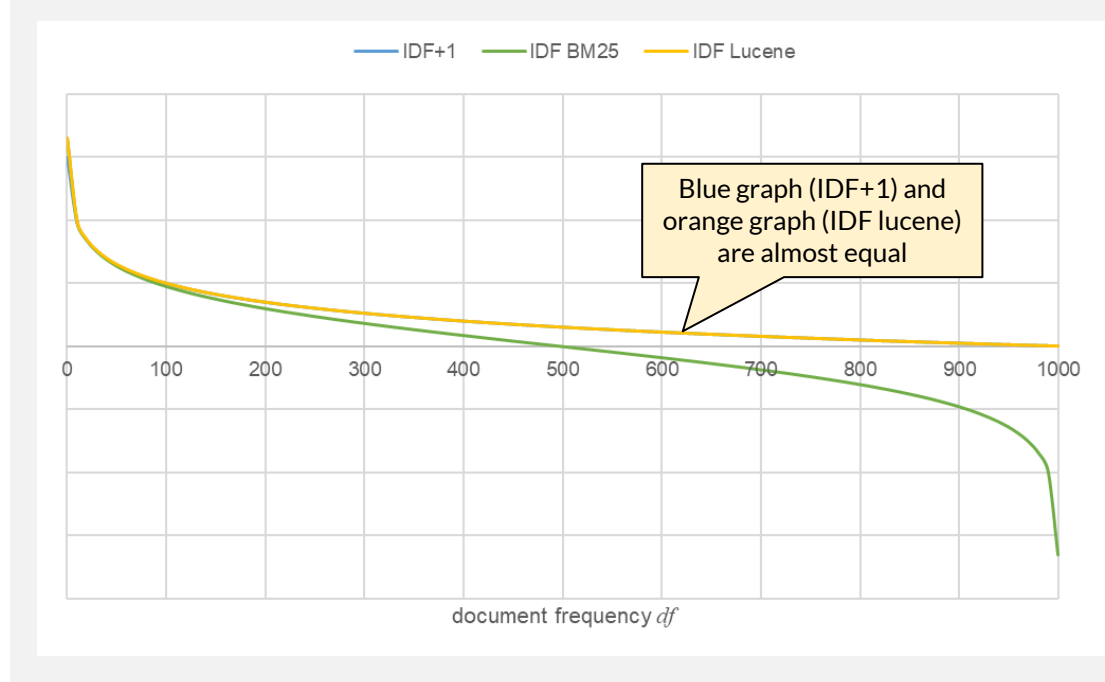
$$= \log \left(\frac{N + 1}{df(t_j) + 0.5} \right)$$

This new *idf*-function, however, yields almost the same values as the original IDF+1 method

- Alternatively, we can avoid negative *idf*-values by assigning a small positive *idf* value to very frequent terms (mostly stop words)
- Finally, BM25 calculates a score for a query Q and a document D_i by summing up the adjusted *tf-idf* values across all query terms q_j :

$$sim_{BM25}(Q, D_i) = \sum_{j=1}^M \log \left(\frac{N - df(t_j) + 0.5}{df(t_j) + 0.5} \right) \cdot \frac{tf(D_i, t_j) \cdot (k + 1)}{tf(D_i, t_j) + k \cdot \left(1 - b + b \frac{|D_i|}{adl} \right)}$$

- Unlike the vector space retrieval model, the *idf*-values are applied only once and query term frequency is not considered. Later in this chapter, we will examine Lucene's scoring function, which expands the above formula with extra components, including query term frequencies and additional term and document weighting.
- In this fundamental formulation, BM25 encompasses three hyperparameters (k, b, adl) that allow fine-tuning the scoring function to match the requirements of the search context.



Observation	How BM25 addresses it	Remarks
Query Term Significance	The vector space retrieval model scores relevance based solely on query terms. More query terms in a document result in higher scores	Because of assumptions about term independence, query terms might not align with semantically relevant terms in the document
Partial Matches	The vector space retrieval model supports partial matches. It ranks partial matches based on the importance of the matched terms, determined by term weights	As above
Document Length	Term saturation varies based on document length; longer documents need a higher number of term occurrences compared to shorter ones	Long documents still face the challenge of ignoring query term positions. Whether query terms appear together in a paragraph is not considered. An effective solution is to divide documents into smaller sections, addressing this concern
Term Specificity	Resolved through the enhanced IDF-based weighting of the relevance scoring function	Specificity varies with context. For instance, consider the query 'car jaguar' where both terms are relatively common. However, in the context of cars, 'jaguar' is much less common than in a broader context
Term Saturation	Implemented using a saturation function on term frequencies which tackles problems related to keyword spamming and prevents excessively frequent terms, like stop words, from dominating the scoring	It is crucial to balance term specificity and term saturation to achieve the best possible outcomes in a search context
Fine Tuning	Offers various hyperparameters for tuning the ranking according to specific search scenarios; default settings are effective in many cases	Refer to discussions on training machine learning methods while validating hyperparameters
Efficiency	The scoring function relies solely on query terms. Given that queries often have less than 5 terms, inverted files ensure high performance	When employing embeddings and vector search, we exchange efficiency for improved semantic relevance evaluation
User Feedback	IDF weights of terms can be fine-tuned using relevance feedback, as demonstrated in the BIR model. Even if the implementation lacks direct support for relevance feedback, we can still modify term weights to adjust scoring based on the feedback	A simple yet effective approach to integrate feedback is through automatic query expansion. Using relevance feedback, additional terms are included that appear frequently in relevant documents but are less common in non-relevant ones
Term Proximity	BM25's relevance scoring lacks direct support for term proximity since it lacks access to term locations within documents	An important scenario involves bi-grams and tri-grams like 'New York' or 'Salt Lake City.' We can enhance our pre-processing to detect common n-grams, which we will study into in the next chapter
Term Dependence	BM25's relevance scoring lacks direct support for term dependence since it treats terms as independent of each other	Common problems involve spelling errors or synonymous forms that convey the same meaning. We will study more advanced approaches in the next chapter

2.5 Literature and Links

General Books on Text Retrieval

- Gerard Salton, Michael J. McGill. **Introduction to Modern Information Retrieval**, McGraw-Hill Book Company, 1983.
- W.B. Frakes and R. Baeza-Yates. **Information Retrieval, Data Structures and Algorithms**, Prentice Hall, 1992.
- Christopher D. Manning, Prabhakar Raghavan, Hinrich Schütze, **Introduction to Information Retrieval**, Cambridge University Press. 2008. <https://nlp.stanford.edu/IR-book/information-retrieval-book.html>
- Karen Sparck Jones and Peter Willet. **Readings in Information Retrieval**. Morgan Kaufmann Publishers Inc., 1997.
- David A. Grossmann and Ophir Frieder. **Information Retrieval: Algorithms and Heuristics**, Kluwer Academic Publishers, 1998/2004.
- Ricardo Baeza-Yates and Berthier Ribeiro-Neto. **Modern Information Retrieval**, ACM Press Books, 1999/2011.
- Sandor Dominich. **Mathematical Foundations of Information Retrieval**, Kluwer Academic Publishers, 2001.
- S. Büttcher, C. Clarke, G. Cormack. **Information Retrieval - Implementing and Evaluating Search Engines**. MIT Press 2010.

Selected Articles on Retrieval models

- G. Salton, A. Wong, and C. S. Yang, **A Vector Space Model for Automatic Indexing**, Communications of the ACM, 1975 (Article in which a vector space model was presented) <https://doi.org/10.1145/361219.361220>
- Robertson, S. E.; Jones, K. Spärck, **Relevance weighting of search terms**. Journal of the American Society for Information Science, 1976. <https://doi.org/10.1002/asi.4630270302>
- Spärck Jones, K., **A Statistical Interpretation of Term Specificity and Its Application in Retrieval**. Journal of Documentation, 1972. <https://doi.org/10.1108/eb026526>
- Porter, M.F. (1980), **An algorithm for suffix stripping**, Program: electronic library and information systems, Vol. 14 No. 3. <https://doi.org/10.1108/eb046814>
- Wikipedia on Zipf's law, https://en.wikipedia.org/wiki/Zipf's_law
- Stephen Robertson and Hugo Zaragoza, **The Probabilistic Relevance Framework: BM25 and Beyond**, Foundations and Trends in Information Retrieval, 2009. <http://dx.doi.org/10.1561/15000000019>

Implementations

- Apache Lucene, <https://lucene.apache.org>
- Apache OpenNLP, <https://opennlp.apache.org/>
- Natural Language Toolkit, <https://www.nltk.org/>
- Apache Solr, <https://solr.apache.org/>
- Elasticsearch, <https://www.elastic.co/>
- OpenSearch, <https://opensearch.org/>